CSE 209 — Computer Architecture

Chapter 4 · The Processor: Single-Cycle Datapath & Control

Lesson notes · Exam approach · Practice with solutions · Quick notes (Rifat Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, bit layouts and hand-drawn datapath sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Course Roadmap — all five phases of the RS part, with exam priorities
Lesson 1 — What Happens When an Instruction Runs? — hardware roles, address vs value, lw/add/sw, ALUSrc
Lesson 2 — Instruction Fetch and the Clock Cycle — PC + 4, clocking, single-cycle timing
Lesson 3 — Register Connections and Basic Control Signals — fields → register file, RegDst, MemtoReg, enables
Lesson 4 — ALUOp, ALU Control and Instruction Traces — two-level control, control table, traces
Lesson 5 — beq: Branch Address, Decision and Control — target formula, Zero, PCSrc, beq control row
Lesson 6 — Jump (j): Address Calculation and Datapath — concatenation, reachability, adding j hardware
Lesson 7 — Full Datapath, Clock Timing and Exam Answers — drawing order, timing, short theory answers
Lesson 8A — Adding bne, jr and jal — datapath modification method
Lesson 8B — slt, lui and Modified Instruction Formats — ALU reuse, LUI path, changed field positions
Quick Notes — All Lessons — merged revision notes + key datapath diagrams

> **NOTE** Every lesson follows the same order: **topic & exam importance → worked examples → exam approach → practice with solutions**. All the per-lesson revision notes are merged into **Quick Notes** at the end. Slide numbers refer to **Chapter 4 Part 1** unless stated otherwise; "TF p." refers to the PDF page of the TF question collection.

Course Roadmap

Phase 1 — Single-Cycle Datapath

Source: Chapter 4 Part 1, with TF applications. (Covered in these notes.)

Lesson	What we learn
1	Hardware roles, address versus value, lw/add/sw execution, and ALUSrc
2	Instruction fetch, PC + 4, register reads/writes, clock edges, and what "single-cycle" means
3	Instruction fields → register-file connections; RegDst, MemtoReg, RegWrite, MemRead and MemWrite
4	ALUOp versus ALU control; derive the control table and trace complete instructions
5	beq: comparison, Zero, branch target, and selecting the next PC
6	j: constructing the jump address and adding the required hardware
7	Full-datapath drawing, longest path, cycle time, separate memories, and single-cycle inefficiency
8	TF modifications: bne, jr, jal, slt, lui, and changed instruction formats

EXAM FOCUS **Main exam targets:** explain a wire or signal, trace an instruction, complete a diagram, and modify a datapath. These recur in the collection.

Phase 2 — Pipelining

Source: Chapter 4 Part 2.

Lesson	What we learn
9	Five stages, overlapping instructions, latency versus throughput, cycle counting and speedup
10	Pipeline registers, what they carry, instruction traces, and control signals travelling with instructions
11	Structural/data/control hazards; identifying dependencies; execution without forwarding
12	Forwarding paths, conditions, and double data hazards
13	Load-use hazards, hazard detection, freezing instructions, and inserting bubbles
14	Timing diagrams, total cycles, and safe instruction rescheduling
15	Branch resolution, flushes, and branch data hazards under different hardware assumptions
16	One-bit/two-bit prediction, prediction accuracy, and branch target buffers
17	Dual issue, scheduling, loop unrolling, and IPC
18	Dynamic scheduling, speculation, remaining processor examples/pitfalls, and mixed pipeline practice

EXAM FOCUS **Highest practice priority:** forwarding, load-use stalls, branch hazards, diagrams, and cycle counting. Always state assumptions such as **where the branch resolves and which forwarding paths exist** — those details can change the answer.

Phase 3 — Memory Hierarchy

Source: Chapter 5.

Lesson	What we learn
19	Locality, hierarchy, hits/misses; concise SRAM, DRAM, flash and disk foundations
20	Direct-mapped caches: blocks, tags, valid bits, and hit/miss traces
21	Address subdivision, byte/word offsets, cache capacity, and total-bit calculations
22	Set-associative/fully associative caches, replacement, and LRU traces
23	Write-through/write-back, write allocation, block-size trade-offs, and miss types
24	AMAT, memory stalls, effective CPI, and multilevel-cache calculations
25	Software locality and blocking; supporting memory-technology calculations
26	Virtual memory, address translation, page tables, page faults, and table-size calculations
27	TLBs, TLB misses versus page faults, and combined TLB/page-table/cache traces
28	Coherence, snooping, directories, consistency, remaining slides, and mixed memory practice

EXAM FOCUS **Highest practice priority:** cache traces, address bits, storage overhead, cache performance, and TLB/page-table questions.

Phase 4 — Implementing the Control Unit

Source: Implementation of the MIPS Control Unit.

Lesson	What we learn
29	Combinational control: truth tables, equations, gates and PLA
30	Brief multicycle-datapath prerequisite, followed by the control FSM and state traces
31	Output/next-state equations and ROM/PLA storage calculations
32	Sequencers, microprogramming, and hardwired versus microprogrammed control

Phase 5 — GPUs and Final Integration

Sources: Graphics and Computing GPUs — Slides, supported by the accompanying chapter.

Lesson	What we learn
33	CPU versus GPU, heterogeneous systems, discrete versus shared-memory designs
34	CUDA: kernels, threads, blocks, grids, memory levels and synchronization
35	GPU hardware organization, mapping work to hardware, throughput, and remaining slide topics
36	Mixed RS TF practice, diagram recall, and targeted repair of weak areas

Lesson 1 — What Happens When an Instruction Runs?

Slides: Chapter 4 Part 1, 3–4 and 13–17

1. Begin with a Small Scenario

Suppose a program wants to:

1. Get a number from memory.
2. Add 5 to it.
3. Save the answer in memory.

```
lw   $t0, 8($s0)
add  $t1, $t0, $s1
sw   $t1, 12($s0)
```

Initially: $s0 = 1000, $s1 = 5, and the memory word at address 1008 = 20. By the end, we expect the word at address 1012 to contain **25**.

Chapter 4 explains **which hardware pieces make this happen and how they connect.**

2. The Hardware You Need to Know First

Think of the CPU as a small workspace:

Hardware	Simple picture	Actual job
Register file	A small set of labelled working boxes	Holds values in registers such as $t0, $s0, $s1
ALU	A calculator	Adds, subtracts, and performs logical operations
Data memory	A large collection of numbered storage locations	Holds data that instructions load or store
Instruction memory	The list of commands	Supplies the instruction to execute
PC	A bookmark in that command list	Holds the address of the current instruction

The **datapath** is the hardware and connections through which these values travel.

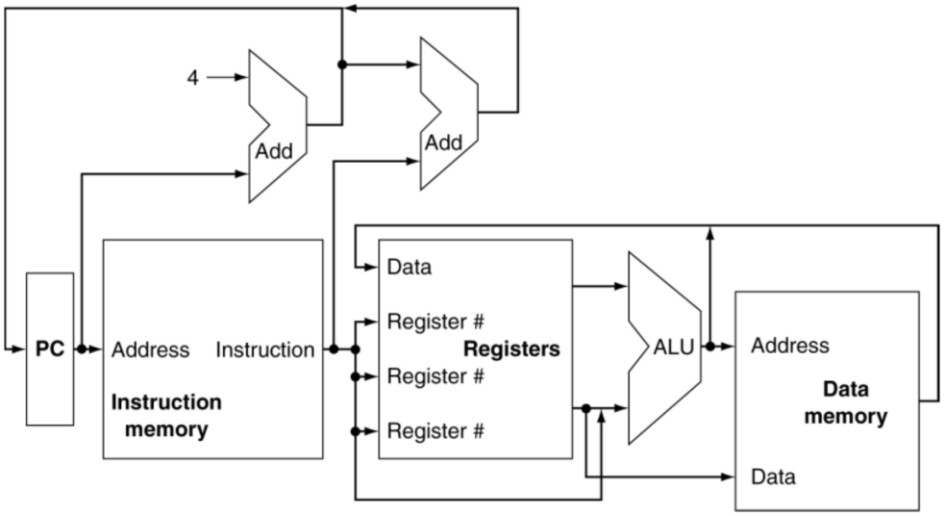


Figure 1.1 — CPU overview: PC, instruction memory, register file, ALU and data memory (slide 4)

A crucial distinction: address versus content

Suppose memory address **1008** holds the word **20**. 1008 tells us **where to look**; 20 is **what we find**.

Similarly, $s0 = 1000 means the register contains the number 1000. In our example, we use that number as an address.

MEMORIZE Keep **location** and **value** separate — it prevents many datapath mistakes.

3. Worked Example: lw Gets Data from Memory

```
lw $t0, 8($s0)
```

> *Start from the address in $s0, move forward 8 bytes, and copy the word there into $t0.*

Step 1 — Read the base address

The register file supplies $s0 → 1000. We need this because the instruction gives an address **relative to $s0**.

Step 2 — Calculate the actual address

The ALU adds 1000 + 8 = 1008. The ALU is acting as an **address calculator** here.

Step 3 — Read the memory word

Data memory receives address 1008 and returns 20.

Step 4 — Put that value into $t0

$t0 ← 20. The complete data path is:

```
Base register + offset → ALU → memory address
                                      ↓
                                 Read memory
                                      ↓
                          Value → destination register
```

TRAP $t0 receives **20**, not 1008.

Another scenario: a negative offset

```
lw $t0, -4($s0)
```

If $s0 = 1000, the address is 1000 − 4 = 996, so the processor loads the word at 996.

The offset is stored in 16 bits, while the ALU uses 32-bit inputs. **Sign extension** widens the offset while preserving its signed value: −4 must remain −4. That is all the sign-extension detail we need for this topic.

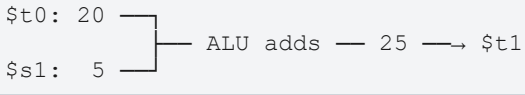
4. Worked Example: add Uses Register Values

After the first load, $t0 = 20 and $s1 = 5. Now execute:

```
add $t1, $t0, $s1
```

The hardware:

1. Reads 20 and 5 from the register file.
2. Sends both to the ALU.
3. Adds them to produce 25.
4. Writes 25 into $t1.



There is no data-memory access. The operands are already in registers. Notice how the ALU's role changes:

- For lw, its answer is a **memory address**.

- For add, its answer is the **value we want to keep**.

EXAM FOCUS Questions ask about the ALU's purpose for different instruction classes — e.g. TF p. 84, Q3(b).

5. Worked Example: sw Sends Data to Memory

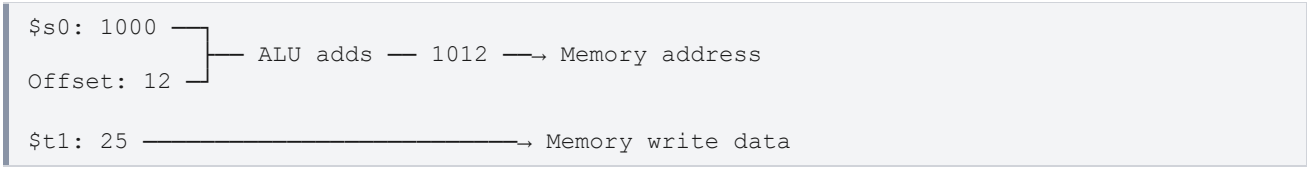
Now $t1 = 25 and $s0 = 1000. Execute:

```
sw $t1, 12($s0)
```

> *Copy the value in $t1 into the memory word located 12 bytes after the address in $s0.*

Two pieces of information must reach memory:

Information	Source	Value
Where to write	ALU computes $s0 + 12	1012
What to write	Register $t1	25



The result is **Memory[1012] ← 25**.

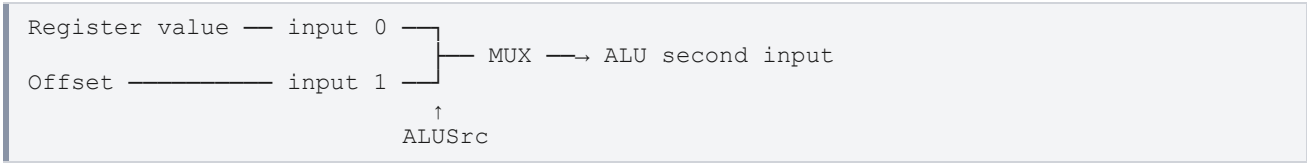
MEMORIZE **sw does not write a register.** It reads registers and writes memory. Reading $t1 does not empty it — $t1 still contains 25.

6. Why a Mux Becomes Necessary

Look at what the ALU needed as its **second input**:

Instruction	Second ALU input
add	register value
lw	offset
sw	offset

One ALU must support all three instructions, so we need a switch that chooses the correct second input. That switch is a **multiplexer (mux)**.



ALUSrc is the control signal telling this mux which input to select. Using the slides' convention:

Instruction	What the ALU needs	ALUSrc
add	Register value	0
lw	Offset	1
sw	Offset	1

MEMORIZE **A control signal tells hardware what to select or enable.** It is not itself the number being calculated. ALUSrc = 1 means "select the offset" — it does **not** mean "send the number 1 to the ALU."

The remaining control signals are built from similar decisions in the next lessons, which makes the control table something you can **reason out** instead of blindly memorize.

7. Hardware Diagrams from the Slides

★ R-format instruction hardware (slide 15)

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

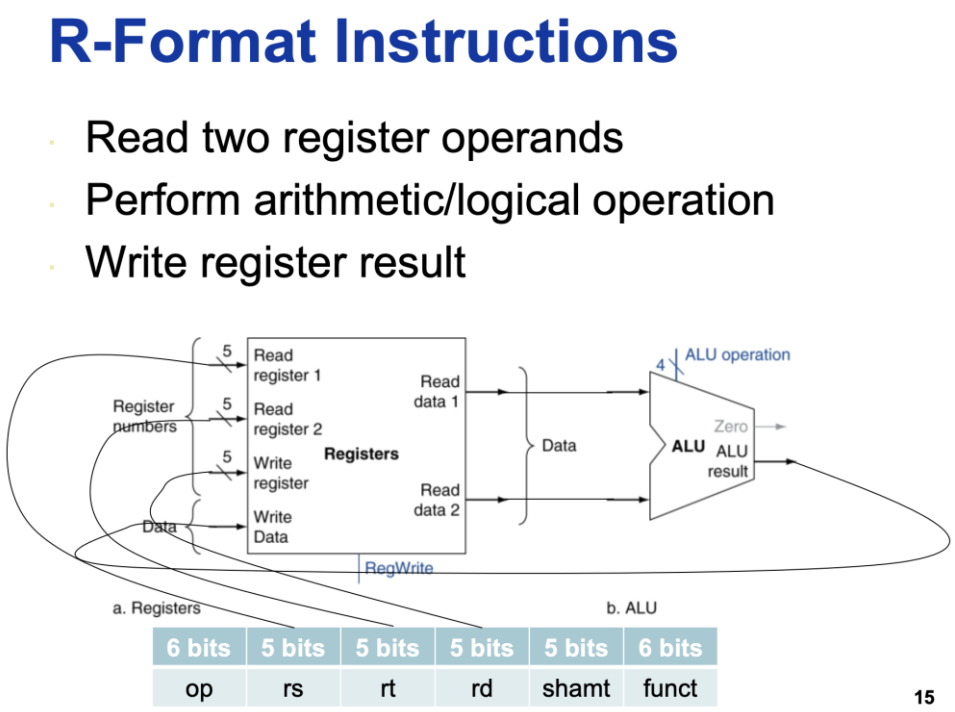


Figure 1.2 — R-format hardware: register file and ALU (slide 15)

★ Load/store hardware (slide 17)

- Read register operands
- Calculate address using the 16-bit offset — use the ALU, but sign-extend the offset
- Load: read memory and update register
- Store: write register value to memory

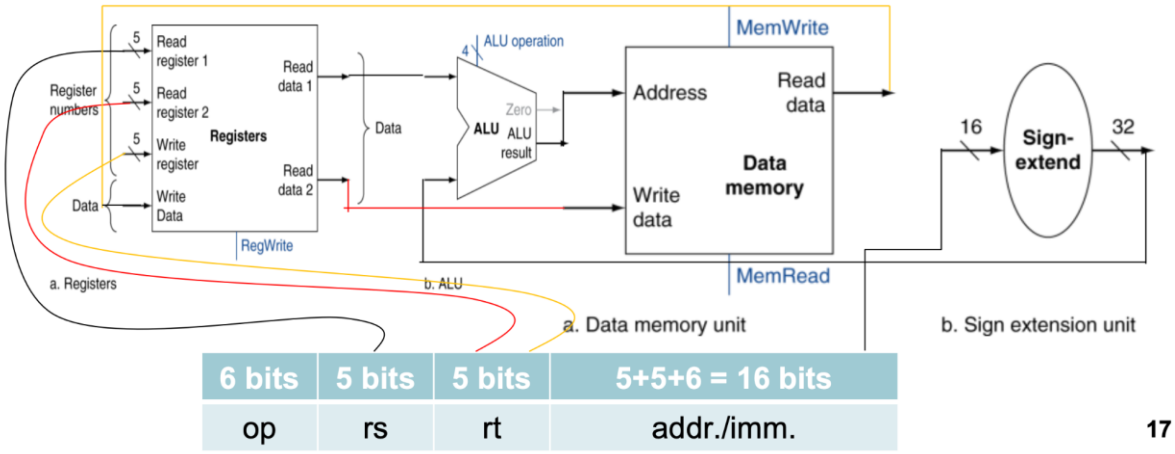
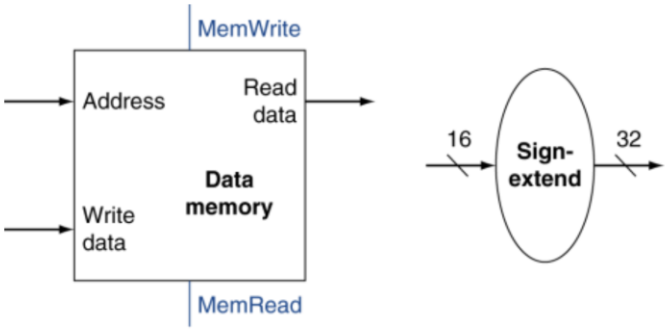


Figure 1.3 — Load/store hardware: red = store path, yellow = load path (slide 17)

Data memory and sign extension (slide 16)



a. Data memory unit b. Sign extension unit

6 bits	5 bits	5 bits	5+5+6 = 16 bits
op	rs	rt	addr./imm.

Figure 1.4 — Data memory unit and sign-extension unit; I-format fields (slide 16)

Instruction fetch is common to all instructions — see **Figure 2.1** in Lesson 2.

Practice — Lesson 1

Constructed TF/CT-style question, based on the instruction-path and control-signal question types in the collection.

Initially: $s0 = 1200, $s1 = 9, Memory[1196] = 31. Execute:

```
lw   $t0, -4($s0)
add  $t1, $t0, $s1
sw   $t1, 8($s0)
```

(a) For each instruction, state the two ALU inputs and the ALU output. Say whether the output is an **address** or an **arithmetic result**.

(b) State the final values of $t0, $t1, and Memory[1208].

(c) Give ALUSrc for each instruction, with a short reason.

(d) A student says: "For sw, the ALU adds $s0 and $t1, then stores the answer." Explain the mistake.

✓ Solution

Instruction	ALU inputs	ALU output
lw $t0, -4($s0)	1200 and −4	1196 — **address**
add $t1, $t0, $s1	31 and 9	40 — **arithmetic result**
sw $t1, 8($s0)	1200 and 8	1208 — **address**

(b) $t0 = 31, $t1 = 40, Memory[1208] = 40.

(c) lw: ALUSrc = 1 (offset); add: ALUSrc = 0 (register value); sw: ALUSrc = 1 (offset).

(d) The statement is wrong. The ALU adds the **$s0 address and the sign-extended offset** — here 1200 and 8. The value in $t1 goes separately to memory's **write-data input**; the ALU calculates only the destination address.

Lesson 2 — Instruction Fetch and the Clock Cycle

Slides: Chapter 4 Part 1, 9–13 and 33

EXAM FOCUS PC + 4 and single-cycle clock length are directly asked in **TF Q8(a), PDF page 7**. Clocking basics are the short prerequisite that makes those answers understandable.

1. Worked Example: How Does the CPU Find an Instruction?

Suppose instruction memory contains:

Starting address	Instruction
400	lw $t0, 8($s0)
404	add $t1, $t0, $s1
408	sw $t1, 12($s0)

Initially **PC = 400**. The **PC contains an address**, not the instruction itself.

Step 1 — Send the PC value to instruction memory

PC: 400 → instruction memory's address input. Instruction memory supplies the instruction stored there (lw $t0, 8($s0)). This is called **instruction fetch**.

Step 2 — Calculate where to go next

A separate adder calculates 400 + 4 = 404. For this ordinary sequential instruction, 404 is the next PC value.

Step 3 — Execute the instruction and update state

During the cycle, the processor performs the load. At the completing clock edge, the destination register receives the loaded value and PC advances to 404. The next cycle therefore fetches add $t1, $t0, $s1.

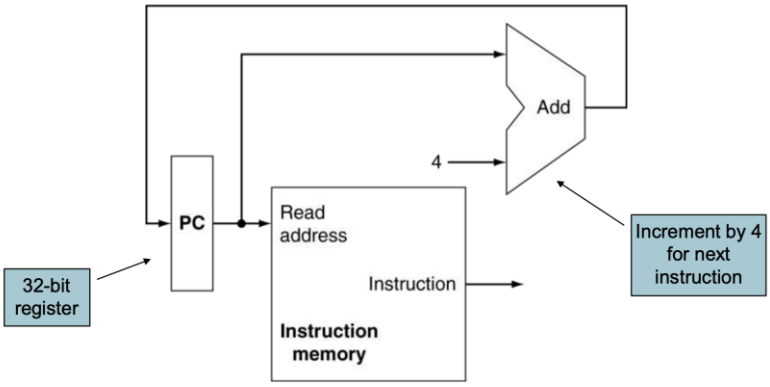


Figure 2.1 — Instruction fetch, common for all instructions (slide 13)

NOTE The feedback path does **not** make PC continuously race through 400, 404, 408, … PC is a register: it holds its current value until the appropriate clock edge.

2. Why Exactly "+4"?

These MIPS instructions are **32 bits = 4 bytes** long, and memory is **byte-addressed**. Thus, the instruction beginning at address 400 occupies:

```
 400   401   402   403
└──── one 32-bit instruction ────┘
```

The next instruction begins at 404. Therefore:

$$PC_{next} = PC + 4$$

MEMORIZE **The reason is instruction size plus byte addressing.** It is not simply "because the PC is 32 bits." A 32-bit PC describes the width of the address it holds; it does not by itself determine the instruction length.

Keep the two addresses separate

For lw $t0, 8($s0) with PC = 400 and $s0 = 1000:

Address	Meaning
400	Where the instruction is stored
404	Where the next sequential instruction is stored
1008	Where the data being loaded is stored

PC + 4 advances through instructions. Base + offset locates data. The fetch adder and the main ALU are separate hardware, so they can calculate these different addresses during the same cycle.

3. What Does the Clock Actually Do?

Imagine a calculator connected to a storage box:

```
Stored inputs → Calculation → Result waiting at storage input
```

The calculator needs time to produce a stable answer. The clock tells the storage box **when to accept that answer**. Two hardware categories matter:

Category	Behaviour	Examples
Combinational logic	Produces outputs from current inputs after a propagation delay	ALU, adder, mux
State elements	Retain stored values; updates follow the clocking/write rules	PC, registers

In the slides' edge-triggered model, registers update on the **rising edge** — when the clock changes from 0 to 1.

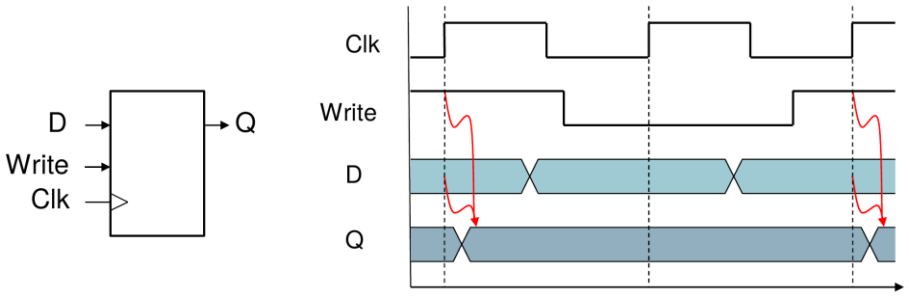


Figure 2.2 — Register with write control: Q updates on the clock edge only when Write = 1 (slide 10)

Example: register write enable

Suppose $t1 currently contains 7, and its proposed new value is 40.

- If register writing is enabled at the write edge, $t1 becomes 40.
- If register writing is disabled, $t1 stays 7.

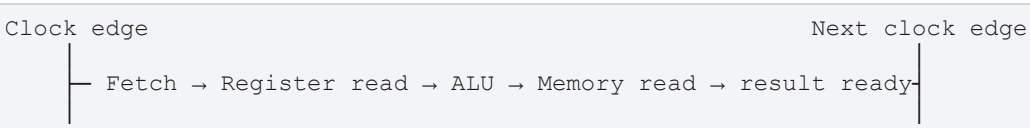
For the register file, **RegWrite** enables the selected register's write.

Reading a register is different: supplying a register number makes its stored value available after the read delay. Reading does not overwrite it. This is why an instruction can **read operands early and write its result later within the same cycle**.

4. How Can lw Do Several Things in One Cycle?

```
Fetch → Read register → Calculate address → Read memory → Write register
```

These are **logical activities**, not necessarily separate clock cycles. In a single-cycle processor, one cycle is long enough for the instruction's entire required path.



The work does not all happen instantaneously. Values travel through dependent hardware operations between clock edges.

MEMORIZE **Single-cycle means one instruction completes in one clock cycle.** It does not mean one hardware operation per instruction.

5. Why Must the Clock Wait for the Slowest Instruction?

Suppose, in a simplified timing example:

Activity	Delay
Instruction fetch	200 ps
Register read	100 ps
ALU operation	100 ps
Data-memory read	200 ps
Register write/setup allowance	100 ps

Then (add skips data memory):

$$T_{lw} = 200 + 100 + 100 + 200 + 100 = \mathbf{700\ ps}$$
$$T_{add} = 200 + 100 + 100 + 100 = \mathbf{500\ ps}$$

With one fixed clock period, the processor must allow at least **700 ps** for these paths.

- **Why not make the cycle 500 ps?** Because lw would reach the ending clock edge before its result was ready. The destination register could capture an incorrect value.
- **What happens to add?** Its result is ready earlier, but it still uses the whole 700 ps cycle.

MEMORIZE The **longest instruction path determines the minimum clock period**. Shorter instructions also use that period, wasting some available time. In the slide's basic implementation, **lw is the critical path**.

Practice — Lesson 2

Constructed numerical scenario; parts (a) and (c) adapt the concepts asked in actual TF Q8(a)(i)–(ii), PDF page 7.

A single-cycle processor executes:

```
lw   $t0, 12($s0)      # stored at address 800
add  $t1, $t0, $s1     # stored at address 804
```

Initially, $s0 = 2000, $s1 = 6, and Memory[2012] = 14.

(a) For the lw, state the instruction address, data address, and next PC. Why is the next PC not 801?

(b) The lw performs five logical activities. Does it take five cycles? Explain briefly.

(c) Suppose lw needs 900 ps and add needs 600 ps. Ignoring additional overhead, what is the minimum fixed clock period, and how long do these two instructions take altogether?

(d) If the proposed write-back value is 20 but RegWrite = 0, does the destination register receive 20 at the clock edge? Explain.

✓ Solution

(a) Instruction address = 800; data address = 2012; next PC = 804. It is not 801 because memory is byte-addressed and each MIPS instruction is 32 bits = 4 bytes long.

> **TRAP** 2000 is only the **base-register value**. The actual memory address includes the offset: data address = R[$s0] + 12 = 2000 + 12 = **2012**.

(b) No — **one cycle**. In a single-cycle processor one instruction completes in one clock cycle, and all five logical activities happen within that cycle.

(c) Minimum fixed clock period = **900 ps**. Together: 2 × 900 = **1800 ps**.

(d) No. A register is written only when RegWrite = 1, so the destination register keeps its old value at the clock edge.

Lesson 3 — Register Connections and Basic Control Signals

Slides: Chapter 4 Part 1, 14–17, 21 and 25–27

EXAM FOCUS **Priority: High.** The TF collection asks both for control-signal values — TF p. 40, Q8(a) — and their functions — TF p. 55, Q2(a). The newer modified-ISA question on TF pp. 4–5 also tests how instruction fields connect to the register file.

Target: given add, lw, or sw, quickly determine the register connections and the six basic control signals.

1. Worked Example: Three Decisions for lw

```
lw $t0, 8($s0)
```

Suppose $s0 = 1000 and Memory[1008] = 42. You already know the result: **$t0 ← 42**. Now think like the hardware designer:

Decision	Required choice	Control
Which register receives the result?	$t0, identified by the instruction's **rt field**	RegDst = 0
What value goes into that register?	The **memory output**, 42	MemtoReg = 1
Should the register actually be written?	Yes	RegWrite = 1

MEMORIZE These signals answer three different questions: **RegDst = where? MemtoReg = what? RegWrite = whether?**

2. Short Prerequisite: Register Number versus Register Value

Suppose $t0 is register number **8**, and currently contains **42**.

- **Register number = 8:** identifies *which* register.
- **Register value = 42:** the data stored inside it.

In the basic MIPS register file:

Input/output	Width	Meaning
Read register 1	5 bits	Number of the first register to read
Read register 2	5 bits	Number of the second register to read
Write register	5 bits	Number of the register to update
Read data 1 and 2	32 bits each	Values obtained from the registers
Write data	32 bits	Value to place in the destination register

Why 5 bits? There are 32 registers, and $2^5 = 32$.

TRAP A wire selecting $t0 carries its **5-bit register number**, not its 32-bit contents.

3. Which Instruction Field Identifies Which Register?

Example A: add

add $t2, $t0, $t1 has the format **add rd, rs, rt**. Therefore:

Field	Register	Role
rs	$t0	First source
rt	$t1	Second source
rd	$t2	Destination

Example B: lw

lw $t0, 8($s0) has the format **lw rt, offset(rs)**.

- rs = $s0: base register to read.
- rt = $t0: destination register to write.

Example C: sw

sw $t0, 8($s0):

- rs = $s0: base register to read.
- rt = $t0: register containing the data to store.
- **No destination register is written.**

> **MEMORIZE** rt is not always a source or always a destination. **Its role depends on the instruction.**

Field positions to memorize

```
R-format:
 31     26 25    21 20    16 15    11 10     6 5        0
┌─────────┬────────┬────────┬────────┬────────┬──────────┐
│ opcode  │   rs   │   rt   │   rd   │ shamt  │  funct   │
└─────────┴────────┴────────┴────────┴────────┴──────────┘

Load/store (I-format):
 31     26 25    21 20    16 15                         0
┌─────────┬────────┬────────┬──────────────────────────────┐
│ opcode  │   rs   │   rt   │          immediate           │
└─────────┴────────┴────────┴──────────────────────────────┘
```

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

Figure 3.1 — MIPS instruction formats (slide 14)

In the standard datapath:

- Instruction[25:21] → Read register 1
- Instruction[20:16] → Read register 2

For lw, the second read output is unnecessary; it does not affect the load result.

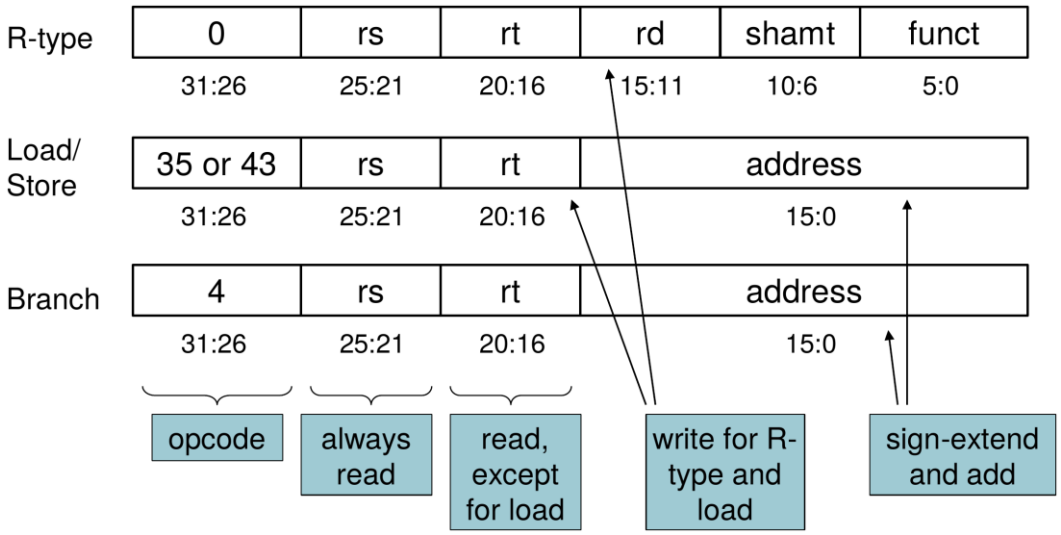


Figure 3.2 — The main control unit: field positions and their roles (slide 25)

4. RegDst — Choose the Destination Register Number

For add, the destination is **rd**. For lw, it is **rt**. A mux chooses which field reaches the register file's **Write register** input:

```
rt: Instruction[20:16] ── 0 ─┐
                             ├── MUX ──→ Write register
rd: Instruction[15:11] ── 1 ─┘
                               ↑
                             RegDst
```

MEMORIZE Slide convention: **RegDst = 0 → rt; RegDst = 1 → rd.** Thus add → 1, lw → 0, sw → X.

Why X for sw?

Because sw does not write any register. Selecting a destination register cannot affect the result when register writing is disabled. **X means don't care: either 0 or 1 works here.** It does not mean that the hardware produces an unknown result.

5. MemtoReg — Choose the Value Written Back

For add $t2, $t0, $t1 the value to write is the **ALU result**. For lw $t0, 8($s0) the value to write is the **memory output**. Another mux selects the register file's **Write data**:

```
ALU result ──────── 0 ─┐
                       ├── MUX ──→ Write data
Memory read data ─ 1 ──┘
                         ↑
                       MemtoReg
```

MEMORIZE **MemtoReg = 0 → ALU result; MemtoReg = 1 → memory data.** Thus add → 0, lw → 1, sw → X (sw writes no register, so the write-data selection does not matter).

6. The Three Enable Signals

These are direct yes/no decisions.

Signal	Set to 1 when…
RegWrite	The instruction writes a register
MemRead	The instruction reads **data memory**
MemWrite	The instruction writes **data memory**

Apply them:

Instruction	RegWrite	MemRead	MemWrite
add	1	0	0
lw	1	1	0
sw	0	0	1

TRAP sw reads registers, but **MemRead = 0**. MemRead refers to **data memory**, not the register file. Likewise, add still fetches an instruction even though MemRead = 0.

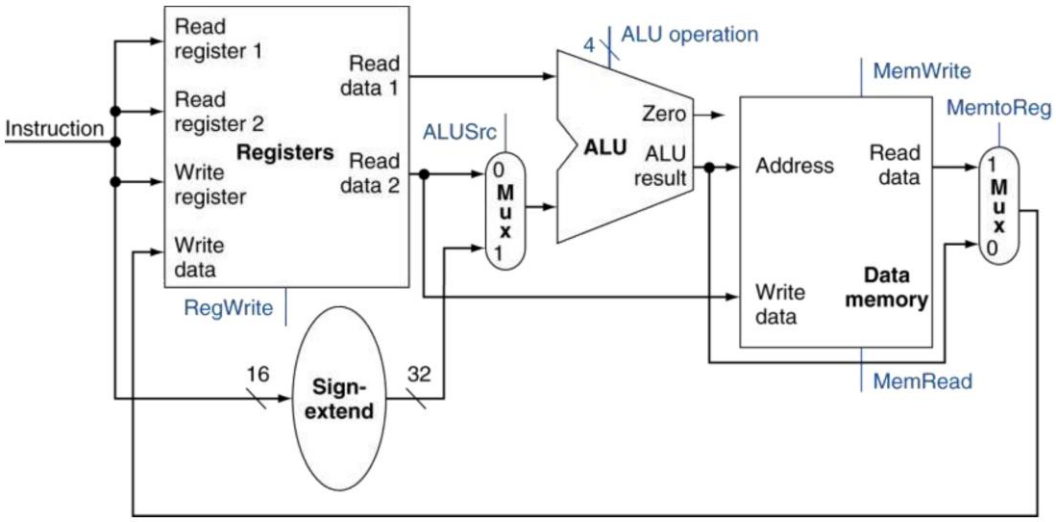


Figure 3.3 — Combined R-type / load / store datapath with its three muxes (slide 21)

Exam Approach

1. Fast method for control-table questions

Use this order:

1. **Writes a register?** Set RegWrite. If no, RegDst and MemtoReg become X.
2. **Reads or writes data memory?** Set MemRead and MemWrite.
3. **Second ALU input?** Register → ALUSrc = 0; offset → ALUSrc = 1.
4. **Destination field?** rt → RegDst = 0; rd → RegDst = 1.
5. **Write-back value?** ALU → MemtoReg = 0; memory → MemtoReg = 1.

Demo: sw $t3, 20($s2)

- Writes a register? **No** → RegWrite = 0; RegDst = X; MemtoReg = X.
- Reads memory? **No** → MemRead = 0.
- Writes memory? **Yes** → MemWrite = 1.
- ALU adds base and offset → ALUSrc = 1.

That is sufficient reasoning. You do not need to narrate the entire instruction cycle.

Memorize this table

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite
add	1	0	0	1	0	0
lw	0	1	1	1	1	0
sw	X	1	X	0	0	1

These are the six basic controls. ALUOp and branch-related controls are added in later lessons.

2. Demo: "Explain RegDst and MemtoReg"

EXAM-READY ANSWER RegDst selects the destination register number: 0 selects the rt field and 1 selects the rd field. MemtoReg selects the register write-back data: 0 selects the ALU result and 1 selects the data-memory output.

If asked to illustrate, add the two mux diagrams above.

3. Demo: a changed instruction format

Suppose a question moves the R-type destination field to **bits [20:16]** and the second source field to **bits [15:11]**. **Method:** follow the new field meanings.

- Connect the second source field [15:11] to the second read-register input for that format.
- Connect the destination field [20:16] to the write-register selection path.
- If other instruction formats require different connections, introduce muxes where the choices differ.

TRAP **Do not blindly reuse standard MIPS bit positions in a modified-ISA question.** This is the key to the TF question on pp. 4–5.

Diagrams to practise

- **Slide 25 (Fig. 3.2):** field positions and their roles — memorize.
- **Slide 21 (Fig. 3.3):** combined R-type/load/store datapath — practise identifying the three muxes and their paths.
- **Slide 27 (Fig. 3.4):** signal meanings — use as your control-reference table.
- **Slide 26 (Fig. 7.1):** full controlled datapath — for now, identify these signals; learn the complete drawing after branches.

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

FIGURE 4.16 The effect of each of the seven control signals. When the 1-bit control to a two-way multiplexor is asserted, the multiplexor selects the input corresponding to 1. Otherwise, if the control is deasserted, the multiplexor selects the 0 input. Remember that the state elements all have the clock as an implicit input and that the clock is used in controlling writes. Gating the clock externally to a state element can create timing problems. (See Appendix B for further discussion of this problem.)

Figure 3.4 — The effect of each of the seven control signals (slide 27)

Practice — Lesson 3

Constructed TF-style question, adapted from the control-signal and field-connection question patterns in the collection.

```
add  $t3, $s1, $s2
lw   $t4, 16($s0)
sw   $t3, 20($s0)
```

(a) Identify rs, rt, and the destination register, if any, for each instruction.

(b) Fill in RegDst, ALUSrc, MemtoReg, RegWrite, MemRead and MemWrite for each instruction.

(c) Explain why sw has MemRead = 0 even though it reads two registers, and why its RegDst is X.

(d) During lw, a fault forces **MemtoReg = 0**. Assume all other controls work correctly, $s0 = 1000, and Memory[1016] = 77. What value will be written into $t4? Explain briefly.

✓ Solution

(a)

Instruction	rs	rt	Destination register
add $t3, $s1, $s2	$s1	$s2	$t3 — in rd
lw $t4, 16($s0)	$s0	$t4	$t4 — in rt
sw $t3, 20($s0)	$s0	$t3	**None**

TRAP **Destination register does not automatically mean rd.** lw uses **rt** as its destination; load/store instructions have **no separate rd field** (lw rt, offset(rs) · sw rt, offset(rs)).

(b)

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite
add	1	0	0	1	0	0
lw	0	1	1	1	1	0
sw	X	1	X	0	0	1

(c)

EXAM-READY ANSWER sw **reads values from the register file** (the base address and the store data), but does not read data memory. Therefore, MemRead = 0. Since RegWrite = 0, no register is written, so RegDst is a don't-care.

(d) MemtoReg selects data; it does **not** disable writing.

```
ALU result    = 1000 + 16 = 1016
Memory output = 77
MemtoReg = 0  → select ALU result → 1016
RegWrite = 1  → write that selected value into $t4
```

$t4 = 1016

So the faulty load writes the **address 1016** instead of the memory value 77.

MEMORIZE **MemtoReg** decides **which value** reaches the register. **RegWrite** decides **whether** the register is updated. "The previous value remains unchanged" would be the answer only if **RegWrite = 0**.

Lesson 4 — ALUOp, ALU Control and Complete Instruction Traces

Slides: Chapter 4 Part 1, 23–24 and 28–29

EXAM FOCUS **Priority: High.** The TF collection explicitly asks how ALU control bits are determined from **ALUOp and the R-type function code** — TF p. 55, Q2(b). This also completes the control-table skills needed for datapath questions.

1. Worked Example: Same ALU Operation, Different Instructions

```
lw   $t0, 12($s0)
add  $t2, $t0, $t1
```

Both instructions require addition — lw adds the base address and offset; add adds two register values. But the control unit identifies these instructions differently.

For lw

The opcode identifies a load. A load always needs addition to calculate its address.

```
lw opcode → Main control: ALUOp = 00
          → ALU control: 0010
          → ALU performs ADD
```

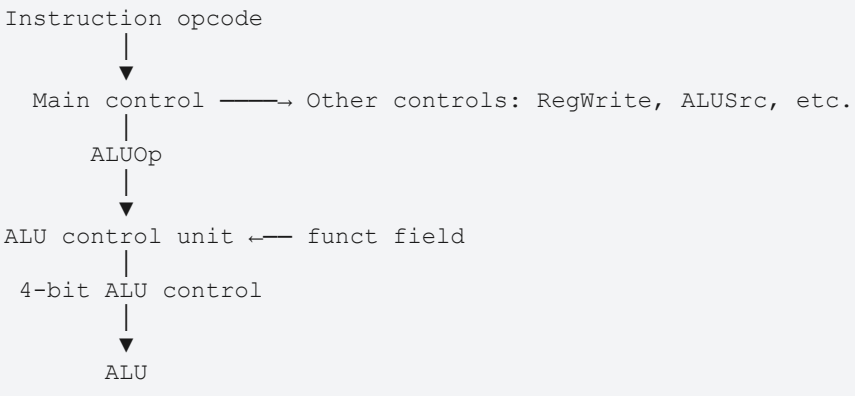
For add

The opcode identifies an **R-type instruction**. However, R-type could mean add, sub, and, or, or slt. The processor must inspect the **funct field** to choose the specific operation.

```
R-type opcode → Main control: ALUOp = 10
funct = 100000 → ALU control: 0010
               → ALU performs ADD
```

MEMORIZE **ALUOp** gives a general instruction **category**; **ALU control** specifies the actual ALU **operation**.

2. The Two-Level Control System



Item	Width in these slides	Purpose
Opcode	6 bits	Identifies the instruction class
ALUOp	2 bits	Tells the ALU control unit how to choose an operation
funct	6 bits	Identifies the specific R-type operation
ALU control	4 bits	Directly selects the ALU operation

TRAP ALUOp = 10 does not mean "subtract" or "add." It means **consult funct**.

3. Memorize These Two Tables

Table A: ALUOp

Instruction class	ALUOp	Meaning
lw, sw	**00**	Perform addition
beq	**01**	Perform subtraction
R-type	**10**	Determine the operation from funct

beq subtracts the two register values to test equality: equal values produce zero. Its complete datapath is covered in Lesson 5.

Table B: Final ALU control (slides' convention)

ALU operation	ALU control
AND	**0000**
OR	**0001**
ADD	**0010**
SUBTRACT	**0110**
Set on less than — SLT	**0111**
NOR	**1100**

Memorization priority: learn the first five first. NOR appears in the slide's ALU-function list, but is not a row in its basic main-control table.

R-type funct values

Instruction	funct
add	**100000**
sub	**100010**
and	**100100**
or	**100101**
slt	**101010**

You already studied instruction encoding in ART. Here, use these values as a lookup table rather than re-deriving them.

4. Worked Examples: Determine the ALU Control

Example A: sw $t0, 20($s0)

1. Identify the class: store.
2. Set **ALUOp = 00**.
3. Therefore, **ALU control = 0010**, meaning addition.

The ALU calculates the address. The value being stored does not determine the operation.

Example B: sub $t2, $t0, $t1

1. Identify the class: R-type.
2. Set **ALUOp = 10**.
3. Read funct = 100010.
4. Therefore, **ALU control = 0110**, meaning subtraction.

Example C: and $t2, $t0, $t1

1. R-type → **ALUOp = 10**.
2. funct = 100100.
3. **ALU control = 0000**, meaning AND.

> NOTE **Useful shortcut:** for the supported R-type instructions, the basic datapath controls remain the same. Only the **funct field and final ALU operation change**.

5. Complete Control Table for the Instructions Covered

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type: add/sub/and/or/slt	1	0	0	1	0	0	0	**10**
lw	0	1	1	1	1	0	0	**00**
sw	X	1	X	0	0	1	0	**00**

For these instructions, **Branch = 0** because none requests a conditional branch.

If a question gives separate columns **ALUOp1** and **ALUOp0**, simply split the two-bit value: ALUOp = 10 → ALUOp1 = 1, ALUOp0 = 0; ALUOp = 00 → ALUOp1 = 0, ALUOp0 = 0.

Exam Approach

1. Question type: "Explain how ALU control is generated"

> EXAM-READY ANSWER The main control unit decodes the opcode and generates the two-bit ALUOp signal. For load/store instructions, ALUOp = 00 selects addition. For beq, ALUOp = 01 selects subtraction. For R-type instructions, ALUOp = 10 causes the ALU control unit to decode the funct field and generate the corresponding four-bit ALU control.

Add the relevant tables if the marks justify them. **Diagram to memorize:** the small two-level control diagram above. **Table to memorize:** slide **24**.

2. Question type: "Trace an instruction through the datapath"

Use this fixed answer order:

> MEMORIZE **Fields → register reads → ALU inputs and operation → memory action → write-back → next PC.**

Demo: lw $t4, 16($s0)

Given PC = 400, $s0 = 1000, Memory[1016] = 77.

Step	Exam answer
Fields	rs = $s0, rt = $t4; rt is the destination
Register read	Read $s0 = 1000
ALU inputs	1000 and sign-extended 16, selected by ALUSrc = 1
ALU operation	ALUOp = 00 → ALU control = 0010 → addition
ALU output	1016, the memory address
Memory action	MemRead = 1 → read 77
Write-back	RegDst = 0 selects $t4; MemtoReg = 1 selects 77; RegWrite = 1 writes it
Next PC	404

Final result: $t4 = 77, PC = 404.

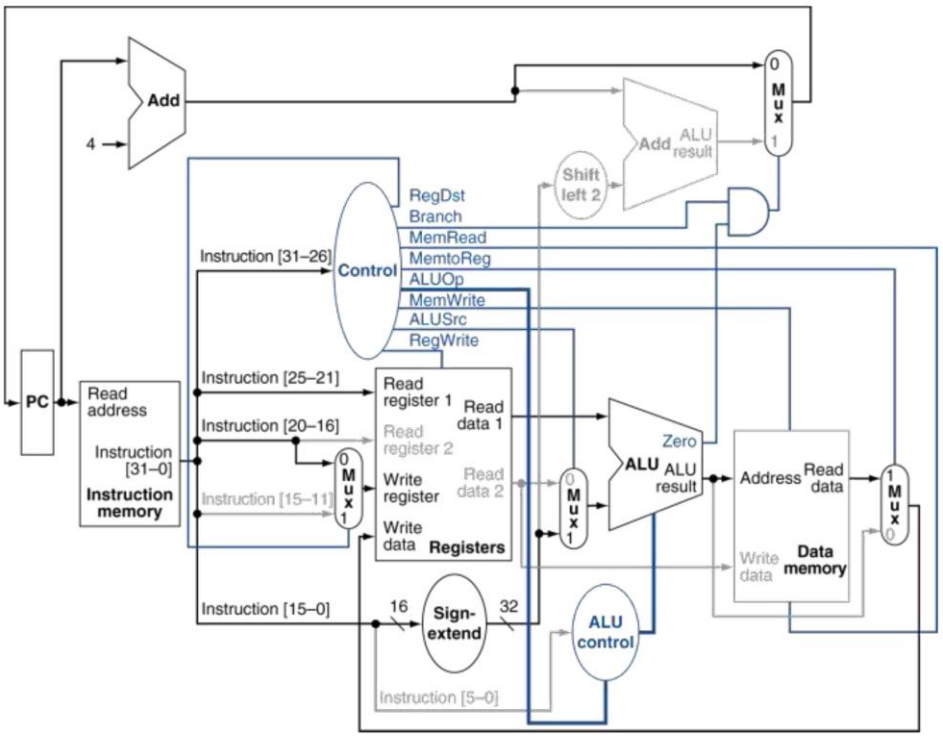


Figure 4.1 — Load instruction: active path through the datapath (slide 29)

Contrast: sub $t2, $t0, $t1

Given $t0 = 50 and $t1 = 12:

```
rs = $t0, rt = $t1, rd = $t2
ALUSrc = 0                → ALU receives 50 and 12
ALUOp = 10, funct = 100010 → subtract
ALU result = 38
No data-memory access
RegDst = 1, MemtoReg = 0, RegWrite = 1 → $t2 = 38
```

This is enough for a short trace answer. Draw or highlight the path only if requested.

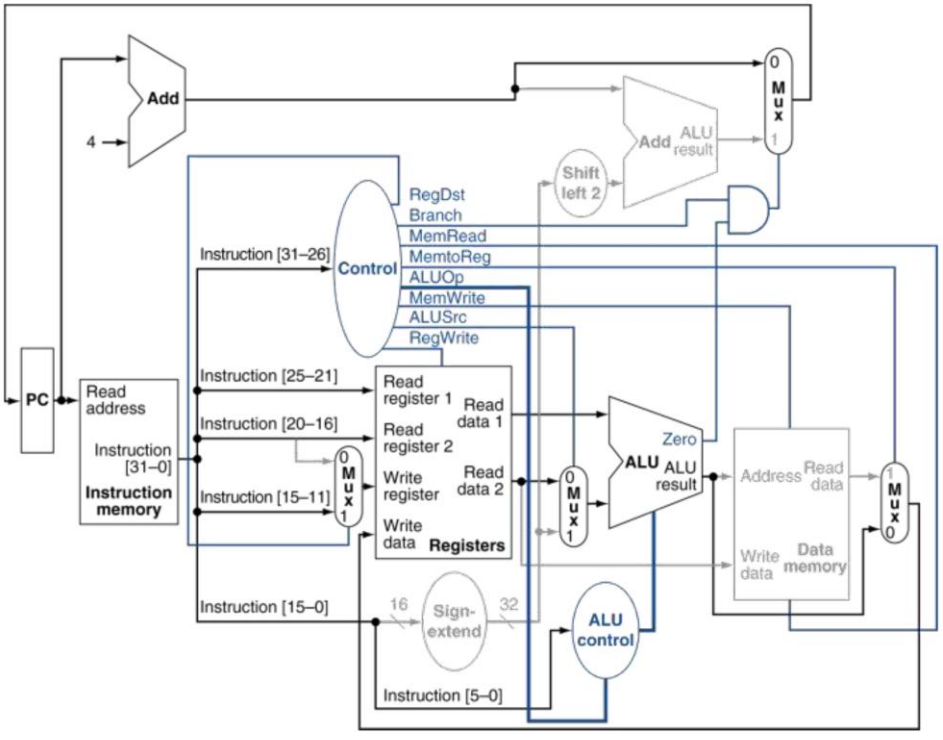


Figure 4.2 — R-type instruction: active path through the datapath (slide 28)

3. Question type: "A control signal is stuck at 0 or 1"

Fast method:

1. Locate what that signal controls.
2. Apply the forced selection or enable.
3. Follow the resulting value to its destination.

Demo: MemtoReg stuck at 0 during lw

Using the previous load: ALU result = 1016, memory output = 77. MemtoReg = 0 selects **1016**. RegWrite remains 1, so **$t4 ← 1016**.

Contrast: RegWrite stuck at 0 during lw

The correct memory value may reach the register-file input, but writing is disabled: **$t4 retains its old value**.

> MEMORIZE **Wrong selection can write the wrong value. Disabled writing preserves the old value.**

What to memorize and practise

Task	Action
ALUOp categories and ALU control codes	**Memorize slides 23–24**
Basic R-type/load/store control table	**Memorize, then practise reconstructing it**
R-type and load instruction paths	**Trace slides 28–29** (Figs. 4.1–4.2)
Full diagram	Identify these paths on **slide 26**; finish learning it after branches

Practice — Lesson 4

Constructed TF-style practice, based on the ALU-control explanation and datapath-tracing questions in the collection.

(a) Complete the table:

Instruction	ALUOp	funct needed to choose the operation?	ALU control
lw $t0, 4($s0)			
sub $t2, $t0, $t1			
or $t3, $t0, $t1			
sw $t2, 8($s0)			

(b) A student says: "Since add and lw both perform addition, both have ALUOp = 00." Correct this statement.

(c) Trace sub $t2, $t0, $t1, given $t0 = 45, $t1 = 17, and PC = 800. State:

- ALU input values and output.
- Destination field and destination register.
- RegDst, ALUSrc, MemtoReg and RegWrite.
- Next PC.

(d) During that sub, a fault forces **ALUOp = 00**, while all other controls remain correct. What value is written into $t2?

✓ Solution

(a)

Instruction	ALUOp	funct needed?	ALU control
lw $t0, 4($s0)	00	No	0010
sub $t2, $t0, $t1	10	Yes — 100010	0110
or $t3, $t0, $t1	10	Yes — 100101	0001
sw $t2, 8($s0)	00	No	0010

(b) Both perform addition, but their ALUOp values differ: add is R-type → **ALUOp = 10** (the ALU control unit then decodes funct = 100000 → 0010); lw → **ALUOp = 00**.

(c)

- ALU inputs = 45 and 17; output = 45 − 17 = **28**.
- Destination field = rd → $t2.
- RegDst = 1, ALUSrc = 0, MemtoReg = 0, RegWrite = 1.
- Next PC = **804**.

(d) ALUOp = 00 is the load/store setting, so ALU control = 0010 and the ALU **adds**. Therefore $t2 is written with 45 + 17 = **62**.

Follow-up question

```
sw $t3, -12($s0)
```

Given $s0 = 2000 and $t3 = 55, state:

1. ALUOp and the final ALU control.
2. The two ALU inputs and resulting memory address.
3. The value written to memory.

Watch the distinction between adding a negative offset and selecting ALU subtraction.

✓ Solution

1. **ALUOp = 00 → ALU control = 0010 (ADD).**
2. Inputs: 2000 and the sign-extended −12. Address = 2000 + (−12) = **1988**.
3. **Memory[1988] ← 55.**

MEMORIZE A negative load/store offset still uses **ALU addition**; the sign-extended operand carries the negative value.

Lesson 5 — beq: Branch Address, Decision and Control

Slides: Chapter 4 Part 1, 18–19, 22 and 30

EXAM FOCUS **Priority: High and repeated.** The TF collection asks for branch datapaths, PC updates and control values — for example, TF p. 17 Q4(a), p. 55 Q2(d), and p. 73 Q8(c).

Targets: calculate the next PC quickly, fill the beq control row, and draw the branch hardware.

1. Worked Example: A Taken Branch

```
beq $s0, $s1, Target
```

Suppose PC = 400, $s0 = 25, $s1 = 25, encoded immediate = 3. Here, **3 is the immediate stored in the instruction**, not the target's address.

> *If the two register values are equal, continue at the branch target. Otherwise, continue at the next sequential instruction.*

There are **two separate calculations**.

Calculation A: should we branch?

The main ALU subtracts 25 − 25 = 0. The ALU's **Zero output becomes 1**, meaning the ALU result is zero. For beq, this means the registers are equal, so the branch is taken.

Calculation B: where is the target?

First calculate the next sequential address, PC + 4 = 404. Then convert the immediate to a byte displacement, 3 × 4 = 12. Finally:

$$\text{Target} = 404 + 12 = \mathbf{416}$$

Since the registers are equal:

$$\mathbf{PC_{next} = 416}$$

MEMORIZE The **main ALU** produces the comparison result, **0**. A **separate branch-target adder** produces the address, **416**.

2. Same Instruction, Unequal Registers

Now change only $s0 = 25, $s1 = 20. The ALU computes 25 − 20 = 5. Therefore **Zero = 0**, the branch is **not taken**, and **next PC = 404**.

The branch-target hardware still calculates 416, but that address is **not selected**.

Register values	ALU subtraction	Zero	Target address	Actual next PC
25 and 25	0	1	416	**416**
25 and 20	5	0	416	**404**

TRAP A question asking for the **branch target** is different from one asking for the **next PC**.

3. Memorize the Branch Formula

$$\mathbf{Branch\ target = PC + 4 + (\ SignExtend(imm16) << 2\ )}$$

For ordinary numerical questions, use:

$$\textbf{Target = PC + 4 + 4 × signed immediate}$$

Then choose:

$$PC_{next} = Target, \text{ if } R[rs] = R[rt]$$
$$PC_{next} = PC + 4, \text{ if } R[rs] \neq R[rt]$$

Why multiply by 4?

The branch immediate expresses a displacement in **instruction words**. Addresses count **bytes**, and one instruction occupies four bytes. Compare this with the load/store rule:

Instruction	Address calculation
lw, sw	Base register + signed immediate
beq	PC + 4 + **4 × signed immediate**

MEMORIZE Load/store offset: **no ×4**. Branch offset: **×4**.

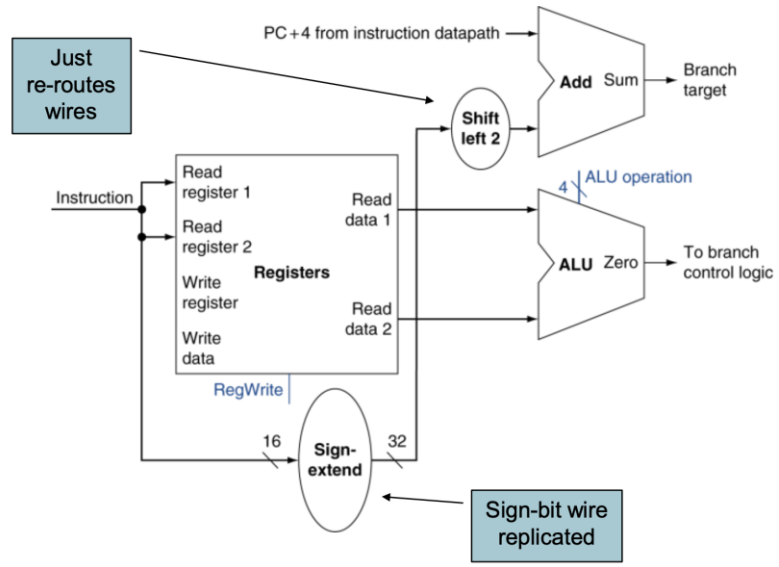


Figure 5.1 — Branch hardware: register comparison in the ALU, sign-extend + shift-left-2 + target adder (slide 19)

4. Branch Control: Branch versus PCSrc

The **Branch** signal says that this instruction is a conditional branch. It does not tell us whether the two values are equal — the ALU's **Zero** output tells us that. For the basic beq datapath:

$$\textbf{PCSrc = Branch AND Zero}$$

PCSrc controls the mux choosing the next PC:

```
[Main control unit] ──(Branch)──┐
                                ├── AND gate ──(PCSrc)──→ [Next-PC mux select]
[ALU Zero flag] ────(Zero)──────┘
```

The multiplexer selects:

- **PCSrc = 0 (default):** next PC = PC + 4 (continue sequentially).
- **PCSrc = 1 (branch taken):** next PC = branch target address.

Situation	Branch	Zero	PCSrc	Next PC
beq, equal registers	1	1	1	Target
beq, unequal registers	1	0	0	PC + 4
Non-branch instruction, even if ALU result is zero	0	1	0	PC + 4

TRAP For an **untaken beq, Branch is still 1**. PCSrc becomes 0 because Zero is 0. Wrong reasoning to avoid: *"The registers are not equal, so the branch is not taken, meaning Branch = 0."*

Case 3 — non-branch instruction, no jump: an instruction like sub $t0, $s1, $s1 yields 0, setting Zero = 1. But because Branch = 0, the CPU does not accidentally branch.

5. Derive the beq Control Row

For beq rs, rt, Target, ask the same questions as before:

Question	Answer	Control
Does it write a register?	No	RegWrite = 0
Does the destination-register selection matter?	No	RegDst = X
Does the register write-back selection matter?	No	MemtoReg = X
Does it read data memory?	No	MemRead = 0
Does it write data memory?	No	MemWrite = 0
What are the ALU operands?	Two register values	ALUSrc = 0
What must the ALU do?	Subtract to compare	ALUOp = 01
Is this a branch instruction?	Yes	Branch = 1

Memorize this row

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
beq	X	**0**	X	0	0	0	**1**	**01**

Final **ALU control = 0110**, selecting subtraction.

NOTE **Why ALUSrc = 0 despite having an immediate?** The main ALU compares the two registers. The immediate goes through separate hardware to calculate the target.

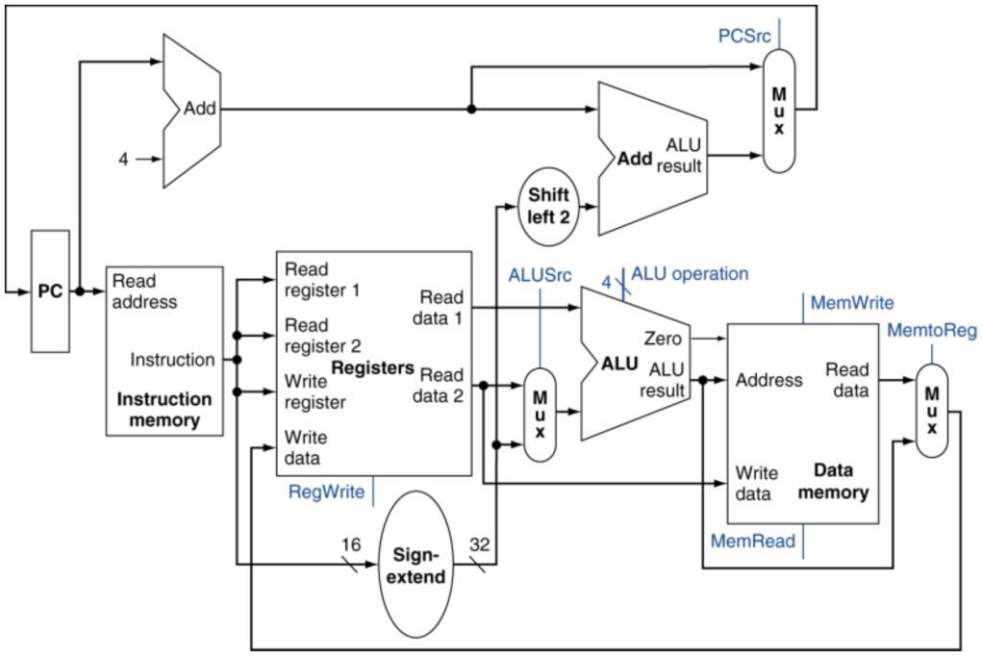


Figure 5.2 — Full datapath: R-type, load/store and branch combined (slide 22)

Exam Approach

1. Numerical question: find the target and next PC

Use this four-line answer:

1. Sequential address = PC + 4
2. Byte displacement = signed immediate × 4
3. Target = sequential address + displacement
4. Compare registers and select target or PC + 4

Demo: backward branch

Given PC = 1000, encoded immediate = −4, registers are equal:

```
PC + 4        = 1004
Displacement  = -4 × 4 = -16
Target        = 1004 - 16 = 988
```

Registers are equal, so **next PC = 988**.

> NOTE **Shortcut:** a negative immediate branches backward relative to PC + 4. If the immediate is given in hexadecimal, interpret it as a **signed 16-bit value** first — e.g. 0xFFFC represents −4.

2. Reverse question: find the encoded immediate

Rearrange the formula:

$$\textbf{Immediate} = (\ \textbf{Target} - (\textbf{PC} + 4)\ ) / 4$$

Demo

Branch instruction address = 800, target address = 840:

$$\text{Immediate} = (840 - 804) / 4 = \mathbf{9}$$

> TRAP Common wrong answer: **10**, obtained by subtracting PC instead of PC + 4.

For a valid ordinary branch encoding, the displacement must be divisible by 4 and the immediate must fit in signed 16 bits.

3. Diagram question: draw the branch datapath

Practise slide 19 (Fig. 5.1), then add the decision gate and next-PC mux shown in the full datapath. Draw these three paths:

```
COMPARISON:
R[rs] ──┐
        ├── Main ALU: subtract ──→ Zero ──┐
R[rt] ──┘                                 ├── AND ──→ PCSrc
                                  Branch ─┘

TARGET CALCULATION:
imm16 → Sign extend → Shift left 2 ──┐
                                     ├── Adder ──→ Branch target
PC + 4 ──────────────────────────────┘

NEXT-PC SELECTION:
PC + 4 ──────────── 0 ──┐
                        ├── MUX ──→ PC input
Branch target ──── 1 ──┘
                    ↑
                 PCSrc
```

Drawing checklist to memorize:

- Two registers feed the subtracting ALU.
- Immediate passes through sign extension and shift-left-2.
- Target adder uses **PC + 4**.
- Branch AND Zero produces PCSrc.
- PCSrc selects sequential address or branch target.

These connections matter more than the exact placement of the boxes.

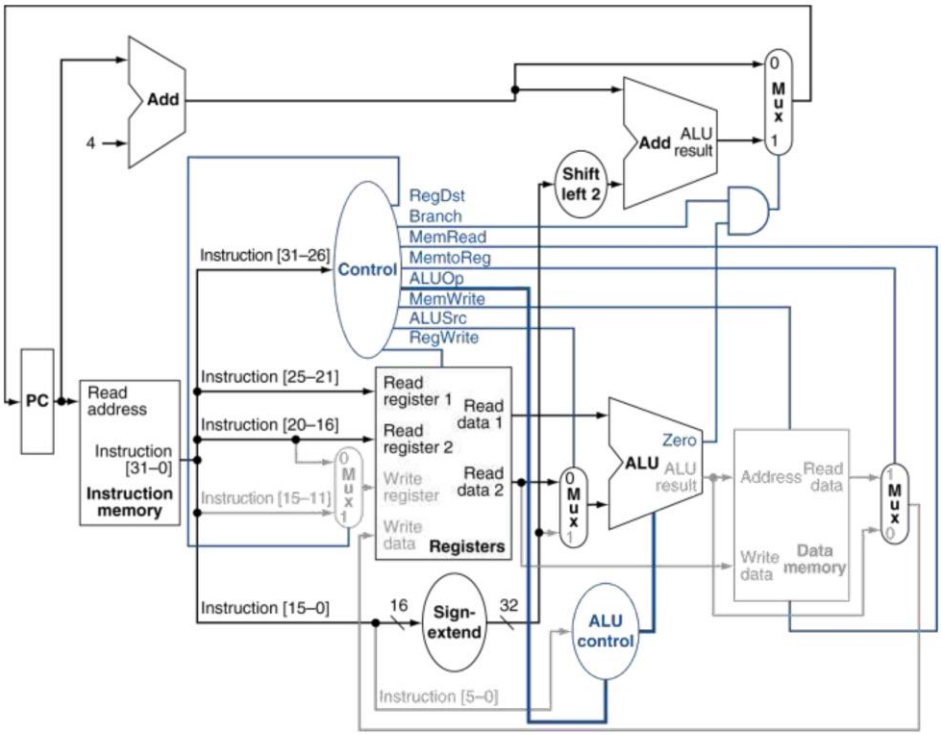


Figure 5.3 — Branch-on-equal: active path through the controlled datapath (slide 30)

4. Theory question: "How is PC updated for beq?"

EXAM-READY ANSWER The ALU subtracts the two register operands. If they are equal, Zero becomes 1. The branch target is calculated by adding the sign-extended immediate shifted left by two bits to PC + 4. PCSrc = Branch AND Zero controls the next-PC mux. If PCSrc is 1, PC receives the branch target; otherwise, it receives PC + 4.

Practice — Lesson 5

Constructed TF-style question, based on the branch-address, PC-update and control questions in the collection.

An instruction at address **1200** is beq $s0, $s1, Target. Its encoded immediate is **−3**, and initially $s0 = 18, $s1 = 18.

(a) Find PC + 4, the byte displacement, the branch target, and the actual next PC.

(b) State the main ALU's two inputs, operation, result, and Zero output.

(c) Give ALUSrc, RegWrite, Branch, ALUOp, final ALU control, and PCSrc.

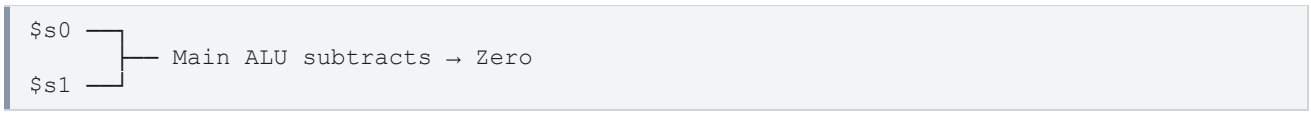
(d) If $s1 changes to 10, what are Zero, Branch, PCSrc, and the next PC?

(e) A different beq is stored at address **2000** and must branch to address **2040**. What immediate must it encode?

✓ Solution

Part	Answer
(a)	PC + 4 = 1204; displacement = −3 × 4 = −12; target = **1192**; next PC = **1192** (since $s0 = $s1).
(b)	Inputs 18 and 18; subtraction; result 18 − 18 = 0 → **Zero = 1**.
(c)	ALUSrc = **0**, RegWrite = 0, Branch = 1, ALUOp = 01, ALU control = 0110, PCSrc = 1.
(d)	Zero = 0, Branch = 1, PCSrc = 0, next PC = **1204**.
(e)	Immediate = (2040 − 2004) / 4 = **9**.

TRAP **Why ALUSrc = 0 for beq (not 1):** the **main ALU compares two register values**, so its second input must come from the register file. The immediate goes to the **separate branch-target adder**, through sign extension and shift-left-2. lw/sw: main ALU uses **base + offset** → ALUSrc = 1. beq: main ALU uses **register − register** → ALUSrc = 0.



Follow-up question

For beq $t0, $t1, Target, suppose $t0 = 30, $t1 = 12, encoded immediate = 5.

1. What are ALUSrc and the main ALU's two input values?
2. Does the immediate 5 enter the main ALU, or the separate branch-target calculation?
3. What are Zero and PCSrc?

✓ Solution

1. **ALUSrc = 0** → main ALU inputs are 30 and 12.
2. Immediate 5 goes through the **separate branch-target calculation**, not the main ALU.
3. 30 − 12 = 18 → **Zero = 0** → **PCSrc = 0**, so the next PC is **PC + 4**.

Lesson 6 — Jump (j): Address Calculation and Datapath

Slides: Chapter 4 Part 1, 31–32

EXAM FOCUS **Priority: Very high.** Adding jump hardware is explicitly asked in **TF PDF page 7, Q8(b)**. Jump-related datapath modifications also appear on TF pages **9, 21, 42 and 78**.

Targets: calculate a jump address, determine whether a target is reachable, and draw the hardware needed to implement j.

1. Worked Example: An Unconditional Jump

```
j Target
```

Unlike beq, j does not compare registers. It always selects the jump target. Suppose PC = 0x20000020 and the 26-bit target field = 0x000000C.

Step 1 — Calculate PC + 4

PC + 4 = 0x20000024. Keep its **upper four bits**, which are the first hexadecimal digit: **2**.

Step 2 — Shift the target field left by 2

This appends two zero **bits**, equivalent to multiplying by 4: 0x000000C × 4 = 0x0000030. The 26-bit field becomes a 28-bit value.

Step 3 — Join the pieces

```
Upper 4 bits   Lower 28 bits
     2            0000030       →  Jump address = 0x20000030
```

$$\mathbf{PC_{next} = 0x20000030}$$

TRAP **Do not add the target displacement to PC + 4.** That is the branch method. Jump uses **concatenation — joining bit fields**.

2. The Formula to Memorize

$$\mathbf{Jump\ address = \{\ (PC + 4)[31:28],\ \ Instruction[25:0],\ \ 00\ \}}$$

The braces mean concatenation:

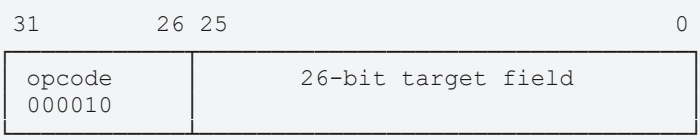
```
┌──────────────────┬──────────────────────────┬──────┐
│ PC+4 upper 4 bits│ 26-bit instruction field │  00  │
└──────────────────┴──────────────────────────┴──────┘
        4          +           26             +  2   = 32 bits
```

Three short reasons worth memorizing

Question	Exam answer
Why append 00?	Instructions are word-aligned, so their byte addresses end in two zero bits.
Why take four bits from PC + 4?	The 26-bit field plus two zeros supplies only 28 bits; four more bits complete the 32-bit address.
Why only a 26-bit target field?	A 32-bit J-format instruction reserves six bits for the opcode.

NOTE **Slide correction:** slide 31 says "top 4 bits of old PC." Use **PC + 4**, as correctly shown in slide 32 and stated in the TF question. The difference matters at an address-region boundary.

J-format — memorize



Opcode for j is **2 decimal = 000010 binary**.

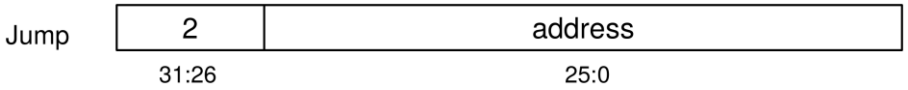


Figure 6.1 — Jump instruction format (slide 31)

3. Fast Comparison: beq versus j

Feature	beq	j
Conditional?	Yes: registers must be equal	No
Address field	Signed 16-bit displacement	26-bit target field
Target calculation	PC + 4 + shifted displacement	Concatenate PC + 4 upper bits, target field and 00
Main ALU needed?	Yes, for subtraction	No
Final decision	Branch AND Zero	Jump control signal

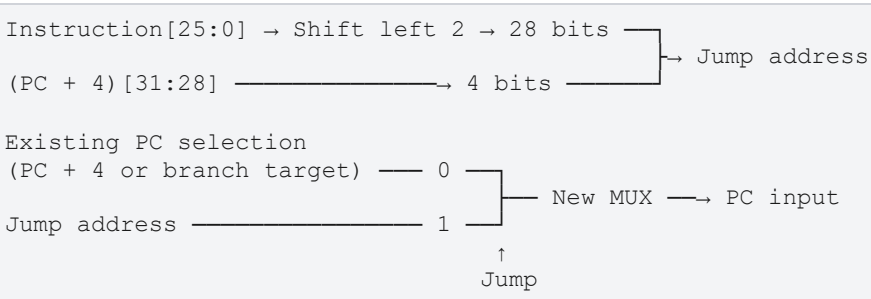
MEMORIZE Branch uses **addition** to PC + 4. Jump uses **concatenation**.

4. Hardware Needed to Add j

Start with the datapath that already chooses between **PC + 4** and the **branch target**. Add one more choice: the **jump target**.

Required additions

1. A path from **Instruction[25:0]** that appends two zeros.
2. Wiring to concatenate **(PC + 4)[31:28]** with that 28-bit result.
3. A **new 2-to-1 mux** before the PC input.
4. A **Jump control signal**, generated by decoding the opcode.



Using slide 32's convention: **Jump = 0** uses the existing sequential/branch result; **Jump = 1** selects the jump address.

MEMORIZE **No new adder is needed for jump-target formation.** Appending zeros and concatenating bits can be implemented with wiring.

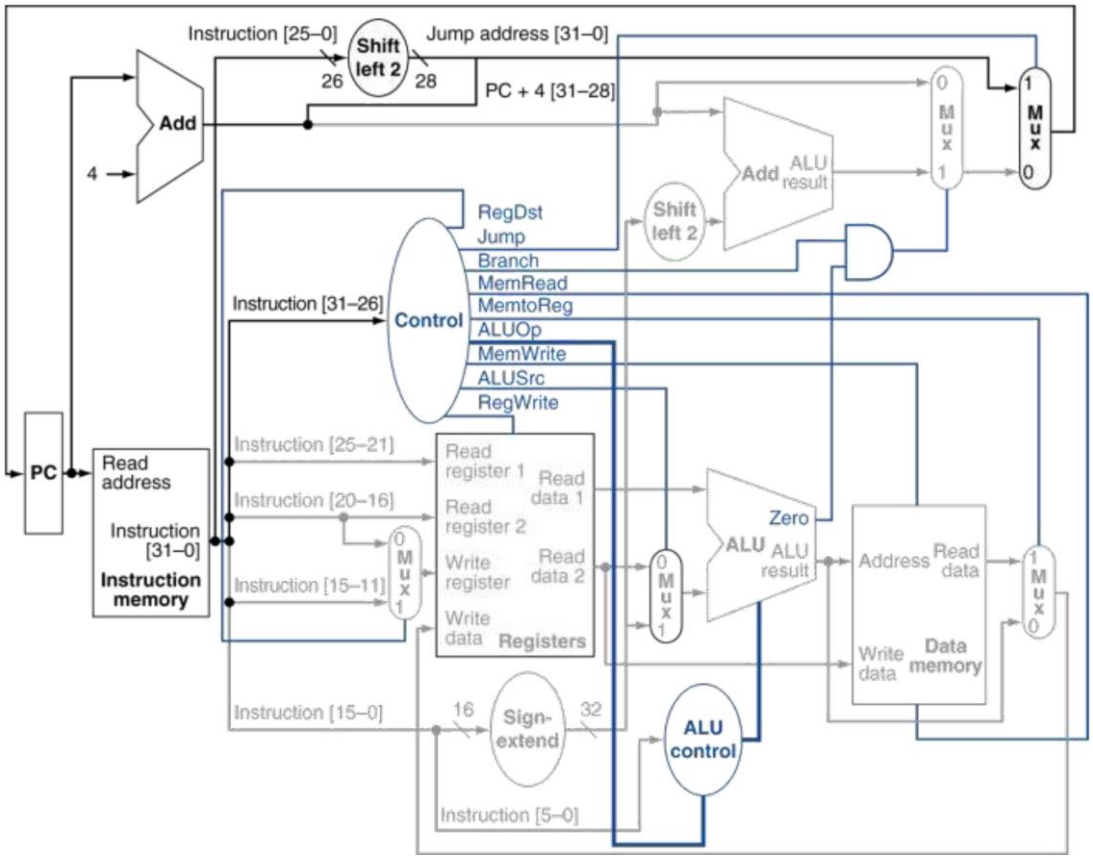


Figure 6.2 — Datapath with jumps added (slide 32) — the base diagram for all modification questions

5. Control Signals for j

j changes the PC but writes neither a general-purpose register nor data memory. A suitable control assignment for the slide-32 datapath is:

Jump	RegWrite	MemWrite	MemRead	Branch	RegDst	ALUSrc	MemtoReg	ALUOp
1	**0**	**0**	**0**	**0**	X	X	X	XX

The ALU result is unused, so ALUSrc and ALUOp do not matter. For the previously studied instructions, **Jump = 0**.

Exam Approach

1. Numerical question: find the jump address

Fast hexadecimal method:

1. Calculate **PC + 4**.
2. Keep its **first hex digit**.
3. Multiply the 26-bit target field by 4.
4. Write that result as **seven hex digits**.
5. Put the saved digit in front.

Demo

```
PC               = 0x80001000
Target field     = 0x0000100
PC + 4           = 0x80001004  → keep 8
Target field × 4 = 0x0000400
Jump address     = 0x80000400
```

TRAP Appending two zero **bits** is not appending two hexadecimal zeros. Two hex zeros would multiply by 256.

2. Reverse question: find the encoded target field

For a reachable, aligned target:

Target field = (Target address AND 0x0FFFFFFF) >> 2

For hand calculation: **remove the target address's first hex digit, then divide the remaining value by 4.**

Demo

PC = 0x20000020, desired target = 0x20000080. The upper hex digits match. Remove the leading 2: 0x0000080 ÷ 4 = 0x0000020. So the encoded target field is **0x20**.

TRAP **Do not subtract PC + 4.** That reverse formula belongs to beq.

3. Reachability question: can one j reach this address?

For a target address to be reachable by a single j instruction:

1. **Word alignment:** the address must end in 00 in binary — hex last digit **0, 4, 8 or C**. Every MIPS instruction occupies 4 bytes, so you cannot jump to the middle of an instruction.
2. **Same 256 MiB block:** the **first hexadecimal digit** (upper 4 bits) of the target must match the **first hexadecimal digit of PC + 4**.

Demo

PC = 0x20000020, target = 0x30000040. PC + 4 = 0x20000024 starts with **2**, but the target starts with **3**.

EXAM-READY ANSWER A single j cannot reach this target because the upper four target bits are inherited from PC + 4 (2), and the instruction cannot set the top digit to 3. j is restricted to the same **256 MiB region as PC + 4**, not an arbitrary 32-bit address.

Why can it only reach 256 MiB?

Recall how the CPU builds the 32-bit destination address:

```
Jump address =  (PC + 4)[31:28]    ||    (target field << 2)
                └─── 4 bits ──┘          └──── 28 bits ─────┘
                  from PC + 4           from the instruction
```

- The instruction supplies **28 bits** of freedom: 2^{28} bytes = 268,435,456 bytes = **256 MiB**.
- The CPU **never modifies the top 4 bits**; it takes them directly from PC + 4.

Because the top 4 bits are locked, memory is partitioned into 16 separate blocks of 256 MiB each:

Block (upper 4 bits)	Memory range
0x0…	0x00000000 – 0x0FFFFFFF
0x1…	0x10000000 – 0x1FFFFFFF
0x2…	0x20000000 – 0x2FFFFFFF
…	…
0xF…	0xF0000000 – 0xFFFFFFFF

A single j instruction **can never cross from one block to another**.

The boundary trap

A common mistake is looking at the top digit of **PC** instead of **PC + 4**. Consider an instruction at the very end of block 0:

```
PC     = 0x0FFFFFFC            (starts with 0)
PC + 4 = 0x0FFFFFFC + 4 = 0x10000000
```

Because the addition carried over into the next hex digit:

- The upper 4 bits are now **1, not 0**.
- Even though the current instruction is located in the 0x0… block, it inherits the top bits of the **0x1… block**.
- It can now only jump to addresses starting with 0x1, not 0x0.

How to jump across blocks?

If a program needs to jump across a 256 MiB boundary (e.g. from 0x20000020 to 0x30000040), it cannot use j. Instead, it uses **jump register (jr)**:

```
lui  $at, 0x3000          # load upper 16 bits
ori  $at, $at, 0x0040     # load lower 16 bits
jr   $at                  # jump to the full 32-bit address in $at
```

jr uses a 32-bit register directly, allowing an unconditional jump to any address in the entire 4 GiB address space.

4. Diagram question: "Add jump support"

For TF Q8(b)-style questions, draw the small added section above and write:

> **EXAM-READY ANSWER** Form the jump address by concatenating the upper four bits of PC + 4, Instruction[25:0], and two zero bits. Add a mux before the PC to select between the existing next-PC value and the jump address. Generate Jump = 1 when opcode = 000010.

Diagram to memorize: slide 32's added jump path (Fig. 6.2). If the question asks for only the added components, you do not need to redraw the entire processor.

Practice — Lesson 6

Constructed/adapted TF-style practice: address calculations are constructed; the hardware question follows the actual jump-extension task on TF PDF page 7.

(a) Given PC = 0x30000020 and 26-bit target field = 0x0000015, find PC + 4 and the jump target.

(b) A j instruction is at 0x30000020 and must jump to 0x300000A0. Find its encoded 26-bit target field.

(c) Can a single j at 0x30000020 jump to 0x400000A0? Explain briefly.

(d) List the hardware/control additions required to add j to a datapath that already supports beq. Is an extra adder required?

(e) Given PC = 0x0FFFFFFC and 26-bit target field = 0, what is the jump target?

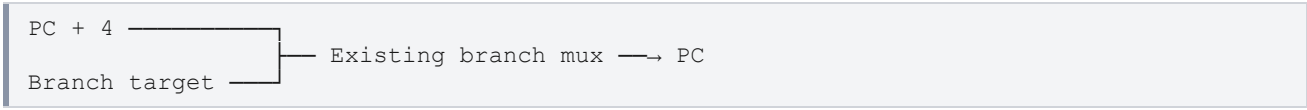
✓ Solution

Part	Answer
(a)	PC + 4 = 0x30000024; jump target = **0x30000054**.
(b)	0xA0 ÷ 4 = **0x28** (field = 0x0000028).
(c)	**No.** PC + 4 starts with 3 but the target starts with 4 — **upper-bit mismatch**. (The target 0x400000A0 **is** word-aligned: its last hex digit is 0.)
(e)	PC + 4 = 0x10000000 crosses the boundary, so the target is **0x10000000**.

TRAP Alignment is **not** the reason in (c). A hex address is word-aligned if its last digit is **0, 4, 8, or C**; 0x400000A0 ends in 0, so alignment is fine.

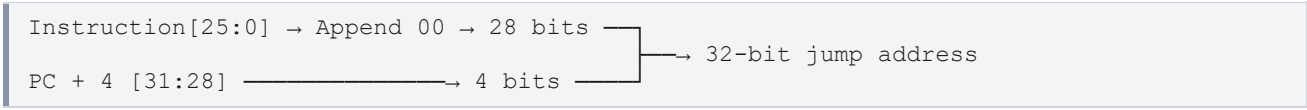
(d) — understanding the question. "You already have a processor that executes beq. What must you add so it can also execute j?" You are being asked to **modify a hardware diagram**, not calculate an address or write assembly code.

What already exists? A processor supporting beq can already choose between two next-PC values:



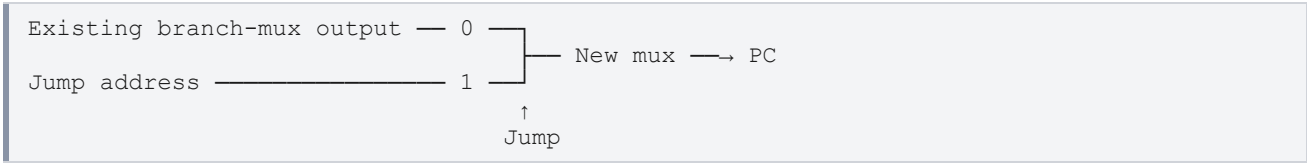
Its existing mux answers "continue normally, or take the branch?" But j needs a **third possible address**: the jump target. Therefore we need to **(1) build that jump address** and **(2) provide a way to select it**.

First addition — build the jump address. Convert the formula (upper 4 bits of PC + 4 | 26-bit target field | 00) into hardware connections:



In an exam diagram, label "Append 00" as **Shift left 2**, matching the slides. **No adder is needed here** — we join bits rather than add numbers.

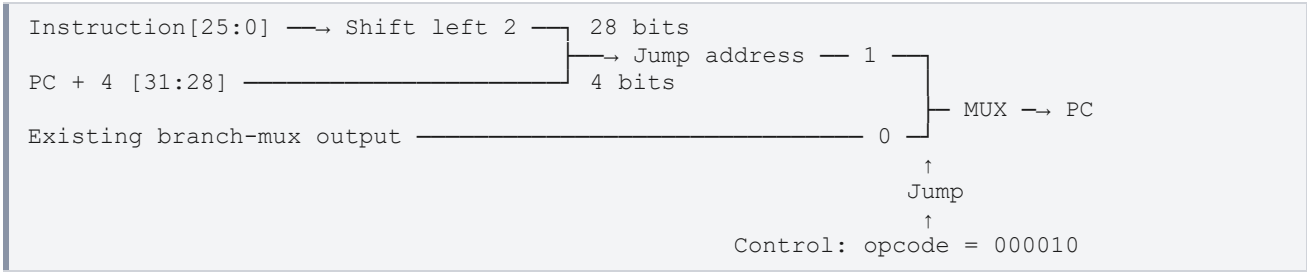
Second addition — select the jump address. Insert one more mux between the old mux and PC:



Jump = 0: use whatever the old datapath selected. **Jump = 1:** use the jump address. Finally, extend the control unit so that it produces **Jump = 1 when the opcode is 000010**.

EXAM-READY ANSWER To implement j, append two zero bits to Instruction[25:0] and concatenate the result with (PC + 4)[31:28] to form the 32-bit jump address. Add a 2-to-1 mux before the PC, selecting between the existing next-PC output and the jump address. Extend the control unit to generate Jump = 1 for opcode 000010. No additional adder is required.

If the question says **"draw,"** include this diagram:



MEMORIZE **Build → Select → Control.** Build: shift and concatenate. Select: add a mux. Control: add Jump. Diagram to memorize: the added jump path on **slide 32** (Fig. 6.2).

Follow-up question

1. What are the **two inputs of the new jump mux**?
2. What value must **Jump** have to select the jump address?
3. Does jump-address formation require an additional adder?
4. Is 0x400000A0 word-aligned, and why?

✓ Solution

1. The two inputs of the new jump mux:

New mux input	Value
0	Previous branch mux's output: PC + 4 or branch target
1	Jump target address: {(PC+4)[31:28], Instruction[25:0], 00}

2. **Jump = 1** selects the target.
3. **No** additional adder is needed.
4. **Yes** — its last hex digit is 0; a final hex digit of 0, 4, 8, or C indicates word alignment.

TRAP The new mux's input 1 is the **32-bit jump target address**, not a 32-bit instruction. The PC receives an **address**; instruction memory uses that address to supply the **instruction**.

Lesson 7 — Full Datapath, Clock Timing and Exam Answers

Slides: Chapter 4 Part 1, 20–22, 26, 32–33

EXAM FOCUS **Priority: High.** The collection asks you to draw datapaths for selected instruction sets and explain clock length, separate memories and single-cycle inefficiency.

This lesson combines the pieces you already know. There is very little new theory; the focus is **how to answer the questions**.

1. Worked Example: Trace One Instruction Through the Full Datapath

```
lw $t0, 12($s0)
```

Given PC = 400, $s0 = 1000, Memory[1012] = 65. Follow the instruction from left to right:

Step	Active path	Value or selection
Fetch	PC → instruction memory	Fetch instruction at 400
Read register	rs → register file	Read $s0 = 1000
Select ALU input	Sign-extended immediate → ALUSrc mux	ALUSrc = 1 selects 12
Calculate address	Register value + immediate → ALU	1000 + 12 = 1012
Access memory	ALU result → data-memory address	Read 65
Select write-back	Memory output → MemtoReg mux	MemtoReg = 1 selects 65
Select destination	rt → RegDst mux	RegDst = 0 selects $t0
Write register	Write-data input → selected register	RegWrite = 1 writes 65
Update PC	PC + 4 → branch mux → jump mux → PC	Branch = 0, Jump = 0 → 404

Final result: $t0 = 65, PC = 404.

TRAP **Exam habit:** when asked to highlight a datapath, trace **both** the instruction's data path **and** the path that produces the next PC. It is easy to finish the register/memory path and forget the PC.

Exam Approach 1 — Drawing the Full Datapath

2. Build the diagram in four layers

Main diagram to practise: slide 32 (Fig. 6.2). Use slide 26 (Fig. 7.1) when the question excludes jump. Don't memorize the drawing as a picture alone — memorize this construction order.

Layer 1 — Place the main blocks

```
PC → Instruction memory → Register file → ALU → Data memory
                                ↑                      │
                                └──── Write-back ──────┘
```

Leave room above for the PC-selection paths.

Layer 2 — Add the three data-selection muxes

Mux	Input 0	Input 1	Output goes to
RegDst	rt	rd	Register-file write-register input
ALUSrc	Read data 2	Sign-extended immediate	ALU second input
MemtoReg	ALU result	Memory read data	Register-file write-data input

Also draw the direct store-data connection: **Register-file Read data 2 → Data-memory Write data.**

MEMORIZE Store data **does not pass through the ALU**.

Layer 3 — Add the PC paths

```
PC → +4 adder → PC + 4

Immediate → Sign extend → Shift left 2 ──┐
                                         ├── Branch-target adder
PC + 4 ──────────────────────────────────┘
```

Then connect:

```
PC + 4 ──────── 0 ──┐
                    ├─ Branch mux ── 0 ──┐
Branch target  1 ──┘                     ├─ Jump mux ──→ PC
                   ↑                     │
            Branch AND Zero        1 ────┘
                                   ↑
                              Jump address
```

The final mux is controlled by **Jump**.

Layer 4 — Add control connections

```
Opcode → Main control
ALUOp + funct → ALU control → ALU
```

Label the register/memory enables and mux controls.

MEMORIZE **Drawing checklist:** main blocks → three data muxes → store-data wire → PC paths → controls. This order prevents most missing-wire mistakes.

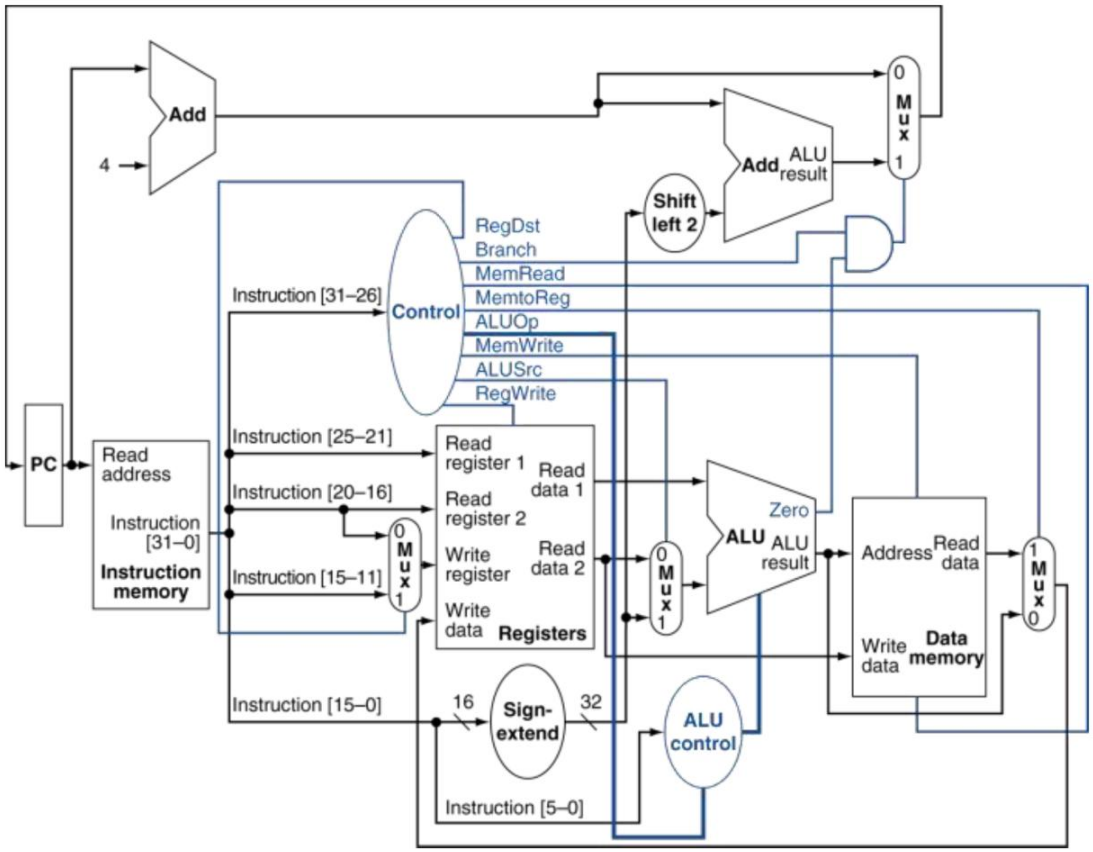


Figure 7.1 — Datapath with control, without jump (slide 26)

3. If the question supports only some instructions

Some TF questions ask for a datapath supporting only a set such as **add, sw, beq**. **Fast method:** start from the full design, then remove hardware needed only by excluded instructions. For this set:

- **No j** → remove jump-target wiring and jump mux.
- **No lw** → no memory-to-register return path is required.
- **Only add writes a register, always rd** → the RegDst mux is unnecessary.
- **Register write-back always comes from the ALU** → the MemtoReg mux is unnecessary.
- **ALUSrc mux remains:** add/beq use a register, while sw uses an immediate.
- **Data-memory write path remains** for sw.
- **Branch hardware remains** for beq.

MEMORIZE **Exam rule:** remove a mux when every instruction that uses its output needs the same input. Check the supported instructions before removing it.

Exam Approach 2 — Clock-Cycle Calculations

4. Worked timing example

Assume these simplified delays:

Component/action	Delay
Instruction memory	200 ps
Register read	100 ps
ALU	150 ps
Data memory	250 ps
Register write/setup allowance	100 ps

Assume mux, control and other delays are ignored, and no other path is longer.

- **lw** needs every listed activity.
- **R-type add** does not access data memory.
- **sw** writes memory but does not write a register.

$$T_{lw} = 200 + 100 + 150 + 250 + 100 = \mathbf{800\ ps}$$
$$T_{add} = 200 + 100 + 150 + 100 = \mathbf{550\ ps}$$
$$T_{sw} = 200 + 100 + 150 + 250 = \mathbf{700\ ps}$$

The fixed single-cycle clock must accommodate the longest path:

$$\mathbf{T_{clock} = 800\ ps}$$

Thus, each instruction takes 800 ps, **including add**.

Total execution time

For 100 instructions:

$$T = 100 \times 800\ ps = \mathbf{80\ ns}$$

TRAP Do not add each instruction's minimum path delay. In this fixed-clock processor, all instructions occupy the same cycle duration.

Maximum clock frequency

$$f_{max} = 1 / T_{clock} = 1 / (800 \times 10^{-12}\ s) = \mathbf{1.25\ GHz}$$

Useful shortcut:

f (GHz) = 1000 / T (ps)

NOTE **Timing caution for exams:** use the delays and assumptions actually supplied. If a question includes mux delays, control delays or additional setup time, include the relevant ones along the path. The example above intentionally omits them.

Exam Approach 3 — Short Theory Answers

5. "What determines the clock-cycle length?"

EXAM-READY ANSWER The clock period must be long enough for the instruction with the longest required path to complete. In the basic single-cycle MIPS datapath, lw is the critical instruction because it requires instruction fetch, register read, ALU address calculation, data-memory read and register write-back.

Memorize the path: **Instruction memory → Register file → ALU → Data memory → Register file**.

6. "Why is single-cycle execution inefficient?"

EXAM-READY ANSWER All instructions use the same clock period, which is determined by the slowest instruction. Instructions with shorter paths, such as R-type arithmetic, still occupy the entire long cycle. Therefore, some instructions leave part of the cycle unused.

This is enough for a short explanation. Add a numerical example if requested: lw needs 800 ps; add needs 550 ps; **both still take 800 ps**.

7. "Why separate instruction and data memories?"

EXAM-READY ANSWER A load/store instruction requires both instruction fetching and data-memory access within the same cycle. Separate instruction and data memories allow these accesses without competing for one memory port.

If the question says *"a single physical memory can be used — agree or disagree?"*, state the assumption:

EXAM-READY ANSWER For the slide's basic single-port memory design, disagree: it cannot support both accesses as required within the single-cycle organization. A suitably designed multiport memory could support both, but that is a different implementation.

MEMORIZE The issue is the **required access capability**, not a universal prohibition on one physical memory.

8. "Why is there a separate PC + 4 adder?"

EXAM-READY ANSWER The main ALU is already needed for the instruction's arithmetic, address calculation or comparison. A separate adder calculates PC + 4 during the same cycle without competing for the main ALU.

For beq, the main ALU also compares registers while another adder calculates the branch target.

Practice — Lesson 7

Constructed/adapted TF-style practice: timing values are constructed; theory and restricted-datapath tasks follow question types verified in the collection.

A simplified single-cycle processor has:

Component/action	Delay
Instruction memory	180 ps
Register read	90 ps
ALU	120 ps
Data memory	220 ps
Register write/setup allowance	90 ps

Ignore other delays and assume no other instruction path is longer than those considered.

(a) Calculate the required path delay for lw, add, and sw. Determine the minimum fixed clock period.

(b) A program executes 10 loads, 20 adds and 5 stores. Find its execution time in nanoseconds.

(c) Explain in two sentences why the processor cannot give add its own shorter cycle while retaining this fixed single-cycle clock.

(d) You must draw a datapath supporting only add and sw. State whether each is required, with a short reason: RegDst mux, ALUSrc mux, MemtoReg mux, branch-target adder, data-memory write path.

✓ Solution

(a)

$$T_{lw} = 180 + 90 + 120 + 220 + 90 = \mathbf{700\ ps}$$
$$T_{add} = 180 + 90 + 120 + 90 = \mathbf{480\ ps}$$
$$T_{sw} = 180 + 90 + 120 + 220 = \mathbf{610\ ps}$$

Minimum fixed clock period = **700 ps**.

(b) Total instructions = 10 + 20 + 5 = **35**.

$$T = 35 \times 700 = 24{,}500\ \text{ps} = \mathbf{24.5\ ns}$$

(c) Every instruction uses the same fixed clock period. That period must accommodate lw, so add also takes **700 ps**.

(d)

Component	Required?	Reason
RegDst mux	**No**	Only add writes a register; always rd.
ALUSrc mux	**Yes**	add uses a register; sw uses an offset.
MemtoReg mux	**No**	Register write-back always uses the ALU result.
Branch-target adder	**No**	Neither instruction branches.
Data-memory write path	**Yes**	Required by sw.

Lesson 8A — Adding bne, jr and jal to the Datapath

Base diagram: Chapter 4 Part 1, slide 32 (Fig. 6.2)

EXAM FOCUS **Priority: Very high — repeated modification questions.** Examples in the TF collection include PDF p. 9 Q3, p. 42 Q1(a), and p. 78 Q2. Lesson 8 is split into **8A: bne, jr, jal** and **8B: slt, lui, and modified instruction formats**.

1. The Method for Every Modification Question

First write what the instruction must do. Then ask:

1. Where does its result come from?
2. Where must that result go?
3. Does that connection already exist?
4. What mux/control must be added?

You usually reuse most of the existing datapath.

2. Adding bne — Branch When Not Equal

Worked example

```
bne $s0, $s1, Target
```

Suppose $s0 = 20, $s1 = 12, PC = 400, immediate = 3. The ALU subtracts:

$$20 - 12 = 8 \Rightarrow \text{Zero} = 0$$

For bne, **unequal registers mean take the branch**. The target calculation is unchanged:

$$\text{Target} = 400 + 4 + 3 \times 4 = \mathbf{416}$$

Only the condition changes. The register reads, subtraction and branch-target hardware already exist.

Exam modification

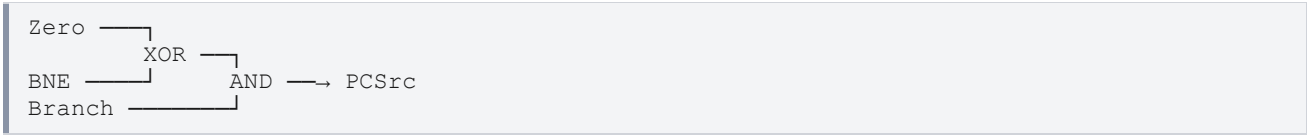
For beq: **Taken = Branch AND Zero**. For bne we need: **Taken = Branch AND (NOT Zero)**.

To support both, add a control bit called **BNE**:

- **BNE = 0** for beq.
- **BNE = 1** for bne.
- Set **Branch = 1** for either branch instruction.

Then:

$$\textbf{PCSrc = Branch AND (Zero XOR BNE)}$$



Instruction	BNE	Selected condition
beq	0	Zero
bne	1	NOT Zero

Control values: bne otherwise uses the same controls as beq — ALUSrc = 0, ALUOp = 01, RegWrite = 0.

TRAP Do not simply invert Zero for the whole processor — that would break beq. The inversion must be **selectable**.

3. Adding jr — Jump to an Address in a Register

Worked example

```
jr $ra
```

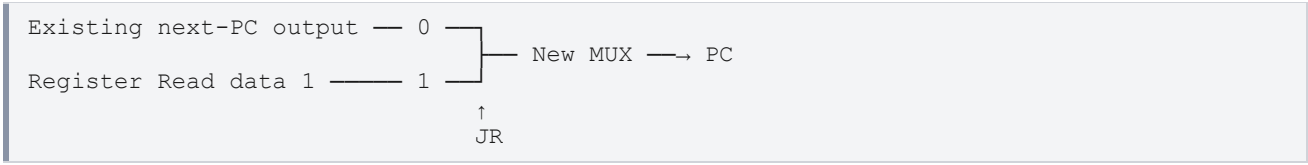
Suppose $ra = 0x00400120. The required operation is:

$$\mathbf{PC \leftarrow R[rs] = 0x00400120}$$

The full address is already stored in the register. **No shifting, concatenation or PC-relative addition is needed.**

Exam modification

The register file already reads rs through **Read data 1**. We need a path from that output to PC. Add another mux after the existing PC-selection path:



- **JR = 0:** normal existing PC selection.
- **JR = 1:** select the register value.

Control decoding: jr is identified by **opcode = 000000** and **funct = 001000**. Therefore, the controller must recognize the funct field as well as the opcode.

Essential controls: JR = 1, RegWrite = 0, MemWrite = 0.

TRAP Although jr uses an R-format encoding, it does **not** use the ordinary R-type RegWrite = 1 setting. The decoder must distinguish it.

4. Adding jal — Jump and Save the Return Address

Worked example

```
jal Function
```

Suppose the instruction is at address 400, and its jump target is 800. In the simplified datapath we are studying: **$ra ← 404** and **PC ← 800**. jal does two things:

1. Jump using the same target calculation as j.
2. Save PC + 4 in register 31, named $ra.

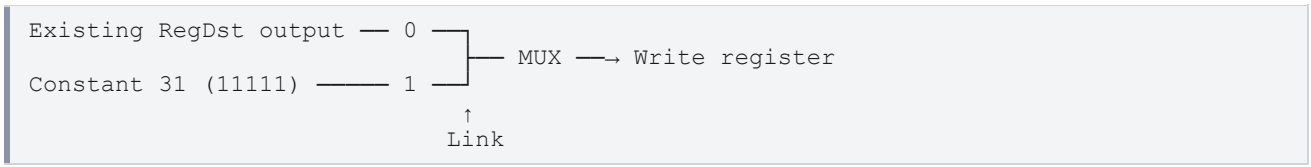
NOTE We use the slides' simplified model **without a delay slot**. If an exam explicitly specifies delayed-branch semantics, the return-address convention must follow that model.

Exam modification

The jump path already exists. What is missing?

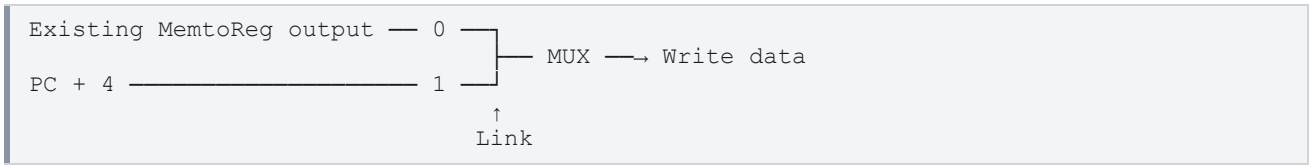
Addition A — Select register 31 as the destination

The existing RegDst mux chooses rt or rd. Add a way to override that result with 31:



Addition B — Select PC + 4 as the write-back value

The existing MemtoReg mux chooses ALU result or memory data. Add another choice:



Essential controls: Jump = 1, Link = 1, RegWrite = 1, MemWrite = 0, JR = 0. For other instructions, set **Link = 0**.

MEMORIZE **No extra adder is required:** PC + 4 is already available. The existing RegDst and MemtoReg selections are don't-cares during jal because the new Link muxes override them.

Exam Approach — What to Write and Draw

5. Compact answers to memorize

Instruction	Required operation	Hardware/control change
bne	Branch when registers differ	Select Zero or NOT Zero using BNE control; preserve existing target calculation
jr	PC ← register rs	Connect Read data 1 to a new next-PC mux; add JR control
jal	$31 ← PC + 4; PC ← jump target	Add destination choice 31 and write-back choice PC + 4; enable Jump, Link and RegWrite

NOTE BNE, JR and Link are names chosen for our added controls. An exam may use different names. Define them clearly and keep the mux labels consistent.

6. Example exam answer: "Implement jal"

EXAM-READY ANSWER Reuse the existing jump-target path. Add muxes to select register 31 as the destination and PC + 4 as the register write-back value. Generate Link = 1 and Jump = 1 for jal, and assert RegWrite. No additional adder is needed because PC + 4 is already computed.

Then draw the two Link muxes, showing their outputs going to the correct register-file inputs.

TRAP Marks come from the **connections and control choices**. A long explanation without the required diagram is not enough when the question asks you to draw.

7. Choosing between mux designs

You may either:

- Add another 2-to-1 mux after an existing mux, as shown above, **or**
- Expand an existing mux to accept more inputs and widen its selection control.

If the question says **"use only 2×1 muxes,"** use the first method. This restriction appears in the question on TF PDF page 9.

Practice — Lesson 8A

Constructed/adapted TF-style practice, based on the repeated "add new instructions" questions.

(a) A bne at PC = 800 has immediate = −2. Its register values are 14 and 9. Find Zero, BNE, PCSrc and the next PC using our design.

(b) For jr $t0, $t0 = 0x00400200. What becomes the next PC? Which register-file output supplies it? Is RegWrite 0 or 1?

(c) A jal at address 1200 jumps to address 2000. In our simplified model, state the new PC, the register written and its value.

(d) A student implements jal by setting only Jump = 1. What additional actions and hardware selections are missing?

✓ Solution

(a) Zero = 0, BNE = 1, PCSrc = 1.

$$PC_{next} = 800 + 4 + 4 \times (-2) = \mathbf{796}$$

(b) Next PC = **0x00400200**; supplied by **register-file Read data 1**, carrying $t0's value; **RegWrite = 0**.

TRAP The source is **Read data 1 (carrying $t0)**, not $ra. jr rs jumps to whichever register is named. $ra is commonly used, but jr is not restricted to it.

(c) PC ← **2000**; $ra (register 31) ← **1204**.

(d) The student implemented the jump but omitted saving the return address. The complete correction is:

1. Add a destination-selection mux to choose **register 31 ($ra)**.
2. Add a write-data mux to choose **PC + 4**.
3. Assert **Link = 1 and RegWrite = 1**, along with Jump = 1.

MEMORIZE jal must select **where to write**, **what to write**, and **enable writing**.

Follow-up questions

1. For jr $s2, which register supplies the next PC, and through which register-file output?
2. During jal, the muxes correctly select register 31 and PC + 4, but **RegWrite = 0**. Does $ra receive the return address? Does the jump still occur if Jump = 1?
3. For jr $t5, which register-file output carries the jump address?

✓ Solution

1. $s2 supplies it, through **Read data 1**:

```
jr $s2
   ↓
rs identifies $s2
   ↓
Register-file Read data 1 outputs R[$s2]
   ↓
JR mux → PC
```

2. **No** — RegWrite = 0 prevents saving the return address. **Yes** — Jump = 1 still causes the jump.
3. **Read data 1.** In jr $t5, $t5 occupies the rs field, so its value appears at Read data 1 and goes through the JR mux to PC.

MEMORIZE **rs → Read data 1; rt → Read data 2.**

Lesson 8B — slt, lui and Modified Instruction Formats

Base diagram: Chapter 4 Part 1, slide 32 (Fig. 6.2)

EXAM FOCUS **Priority: High.** Adding slt and lui appears in the modification question on **TF PDF page 42, Q1(a)**. A changed instruction format appears in the newer question on **TF PDF pages 4–5, Q5(a)**.

The method is the same: **required result → destination → missing connection → control changes.**

1. Adding slt — Usually No New Datapath Hardware

Worked example

```
slt $t0, $s0, $s1
```

Suppose $s0 = 7 and $s1 = 12. slt means **set on less than**:

R[rd] = 1 if R[rs] < R[rt]; otherwise R[rd] = 0

Since 7 < 12, **$t0 ← 1**. If $s0 = 15 and $s1 = 12, $t0 receives **0**.

It writes a Boolean result into a register; it does not branch. For slt, the comparison is signed.

How the datapath executes it

```
Read rs and rt → ALU performs SLT → write result into rd
```

This is already the ordinary R-type route.

RegDst	ALUSrc	MemtoReg	RegWrite	MemRead / MemWrite / Branch / Jump	ALUOp	funct	ALU control
1	0	0	1	0	10	**101010**	**0111**

Exam answer: what must be added?

EXAM-READY ANSWER If the ALU already supports SLT, no new datapath connection is required. Decode funct = 101010 to generate ALU control = 0111, and use the normal R-type controls.

NOTE **Important qualification:** the slides' ALU already lists SLT. If a question explicitly gives an ALU without that operation, its comparison capability must also be added.

MEMORIZE slt follows the **R-type write-back path**, not the branch path.

2. Adding lui — Put an Immediate in the Upper Half

Worked example

```
lui $t0, 0x1234
```

The required result is **$t0 ← 0x12340000**. lui means **load upper immediate**:

```
Immediate:          1234
Append 16 zeros:    1234 0000
```

In bit notation:

R[rt] ← { imm16, 16'b0 }

The lower 16 bits become zero. The instruction does **not** preserve the previous lower half of the register.

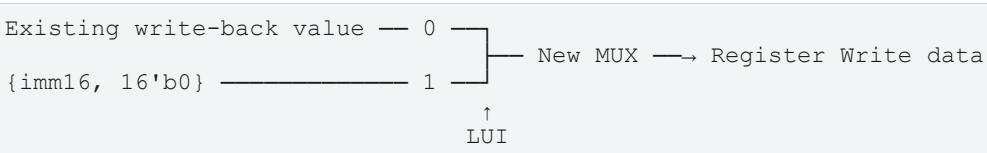
Another example: lui $t1, 0xABCD → $t1 = 0xABCD0000. No data-memory read is involved. The word "load" here does not mean lw.

Hardware modification for lui

The standard write-back path can choose the ALU result or the memory output. We need a third source: **the immediate followed by 16 zero bits**.

```
Instruction[15:0] → Append 16 zero bits → LUI value
```

Add a mux after the existing MemtoReg mux:



This fixed shift can be implemented by wiring. It does not require a general-purpose variable shifter.

Controls for this implementation

Control	Value	Reason
LUI	**1**	Select the constructed LUI value
RegDst	**0**	Destination is rt
RegWrite	**1**	Write the register
MemRead / MemWrite	0	No data-memory access
Branch / Jump	0	Sequential execution
ALUSrc / ALUOp	X / XX	ALU result is unused
MemtoReg	X	New LUI mux overrides its output

If you retain the earlier extensions, set **JR = 0 and Link = 0** for lui. **Control decoding:** lui opcode = **001111**.

Exam-ready answer

EXAM-READY ANSWER Construct the LUI result by concatenating Instruction[15:0] with 16 zero bits. Add a write-back mux input for this value and generate an LUI control signal. Select rt as the destination and assert RegWrite. No data-memory access is required.

3. Modified Instruction Formats — Follow Meaning, Not Familiar Positions

This is where knowing the standard diagram alone is insufficient. A question may rearrange the instruction fields and ask you to connect them to the register file.

Worked example — constructed format

Suppose the question gives:

Format	Bits [25:21]	Bits [20:16]	Bits [15:11]
Modified R-type	rt — second source	rd — destination	rs — first source
Modified I-type	rt — destination for lw	rs — base	Part of immediate

The register-file ports retain their meanings:

- **Read register 1:** first source/base register number.
- **Read register 2:** second source/store-data register number.
- **Write register:** destination register number.

Step 1 — Build a connection table

Register-file input	R-type needs	I-type needs
Read register 1	rs = bits **[15:11]**	rs = bits **[20:16]**
Read register 2	rt = bits **[25:21]**	rt = bits **[25:21]**
Write register	rd = bits **[20:16]**	rt = bits **[25:21]**

For lw, Read data 2 is unused. For a store using this I-format, it supplies the data to store.

Step 2 — Compare each row

- Read register 1 needs **different fields** → add a mux.
- Read register 2 needs **the same field** → direct wire.
- Write register needs **different fields** → add a mux.

Answer: two 5-bit-wide muxes are sufficient for these three register-number inputs.

Step 3 — Draw and label

Define R = 1 for R-type and R = 0 for I-type:



> **TRAP** These wires carry **register numbers**, so they are **5 bits wide — not 32**.

Exam Approach — Fast Decision Rules

Question asks you to…	What to do
Add slt	Check whether ALU supports SLT; use R-type path and funct decoding
Add lui	Construct {imm16, 16'b0} and route it to register write-back
Reconnect a changed instruction format	Make a source-field table for each register-file input
Minimize muxes	Same required field → direct wire; different fields → mux
Use only 2×1 muxes	Add cascaded two-input muxes rather than an unrestricted multi-input mux

What to memorize

- slt: funct **101010**, ALU control **0111**, destination **rd**.
- lui: opcode **001111**, destination **rt**, result **immediate followed by 16 zeros**.
- Register-number wires are **5 bits**; register-data wires are **32 bits**.
- For changed formats, **derive connections from the supplied field meanings**.

Diagram practice

Use **slide 32 (Fig. 6.2)** as the base.

- For slt, trace the ordinary R-type path.
- For lui, practise drawing the added immediate-to-write-back path.
- For changed formats, practise the three register-number input connections independently of the full processor.

> **NOTE** The lui modification above is an exam-extension design, not a diagram printed on slide 32.

Practice — Lesson 8B

Constructed TF-style practice, based on the verified instruction-extension and modified-format question types.

(a) Execute slt $t0, $s0, $s1 given $s0 = −3 and $s1 = 2. State the result, destination field, ALUOp and final ALU control.

(b) Initially $t1 = 0xFFFFFFFF. What is its value after lui $t1, 0x2468? Does the instruction read data memory?

(c) State the missing write-back source and destination-register choice needed to implement lui.

(d) A modified ISA has:

Format	[25:21]	[20:16]	[15:11]
R-type	rd	rs	rt
Load/store	rs	rt	Part of immediate

For **Read register 1, Read register 2 and Write register**, give the required field positions for each format. How many 2-to-1 muxes are needed for these connections? Treat the store's write-register selection as irrelevant.

✓ Solution

(a) Since −3 < 2: **$t0 = 1**; destination field = **rd**; ALUOp = **10**; ALU control = **0111**.

(b) **$t1 = 0x24680000.** The previous contents are overwritten. No data-memory read.

(c) Add **{imm16, 16 zero bits}** as a write-back source. Select **rt** as destination: RegDst = 0, RegWrite = 1.

(d)

Register-file input	R-type	Load/store
Read register 1	rs: [20:16]	rs: [25:21]
Read register 2	rt: [15:11]	rt: [20:16]
Write register	rd: [25:21]	Load destination rt: [20:16]; irrelevant for store

Three 2-to-1 muxes, each 5 bits wide, because each port needs different field positions across the formats.

Quick Notes — All Lessons

The revision notes of every lesson, merged in one place for fast review. Figure numbers point back to the lessons.

L1 · Basic Components and Instruction Paths

The datapath consists of hardware and connections that process and transfer data.

- **PC:** stores the address of the current instruction.
- **Instruction memory:** supplies the instruction.
- **Register file:** stores register values and supplies operands.
- **ALU:** performs arithmetic, logical operations, and address calculation.
- **Data memory:** stores data accessed by load/store instructions.
- **Multiplexer (mux):** selects one input from alternative sources.
- **Control signals:** select datapath inputs or enable actions such as writing.

Instruction	Operation	ALU inputs	ALU output	Final write
add rd, rs, rt	R[rd] ← R[rs] + R[rt]	Two register values	Arithmetic result	Register rd
lw rt, offset(rs)	R[rt] ← Memory[R[rs] + offset]	Base register and sign-extended offset	Memory address	Register rt receives memory data
sw rt, offset(rs)	Memory[R[rs] + offset] ← R[rt]	Base register and sign-extended offset	Memory address	Memory receives register rt's value

Effective address = base-register value + sign-extended offset

The offset is measured in bytes. It is **not** multiplied by 4.

Worked trace — initially $s0 = 1000, $s1 = 5, Memory[1008] = 20:

1. lw $t0, 8($s0) → address = 1008 → $t0 = 20.
2. add $t1, $t0, $s1 → $t1 = 20 + 5 = 25.
3. sw $t1, 12($s0) → address = 1012 → Memory[1012] = 25.

ALUSrc controls the mux selecting the ALU's second operand: **0** = second register value, **1** = sign-extended immediate. Therefore add → 0, lw → 1, sw → 1. Sign extension converts the 16-bit offset to 32 bits while preserving its signed value.

Diagrams: R-format hardware (Fig. 1.2), load/store hardware (Fig. 1.3), data memory & sign-extend (Fig. 1.4).

TRAP

- For lw/sw, the ALU produces an **address**, not the memory contents.
- lw writes a register; sw writes memory.
- sw needs both an address and a value to store.
- add uses two register operands and does not access data memory.
- Reading a register or loading a memory word does not erase the source.
- A mux control value selects an input; it is not the operand value.

L2 · Fetch, Clocking and Single-Cycle Execution

```
PC → Instruction memory → Instruction bits
PC → Separate +4 adder  → Next sequential PC
```

$$PC_{next} = PC + 4$$

Reason: each MIPS instruction is 32 bits = 4 bytes, and memory is byte-addressed. Example: an instruction starting at 400 occupies bytes 400–403; the next starts at 404. **Diagram to practise:** slide 13 (Fig. 2.1).

Instruction address vs data address — for lw $t0, 8($s0), with PC = 400 and $s0 = 1000: instruction address = 400; next sequential instruction address = 404; data address = 1000 + 8 = 1008. The PC + 4 adder is separate from the main ALU.

Clocking

- **Combinational logic:** ALU, adders and muxes produce outputs from inputs after a delay.
- **State elements:** PC and registers retain values.
- In the slides' model, registers update on the **rising clock edge**.
- A register-file write requires **RegWrite = 1**.
- Reading a register does not change its stored value.

Single-cycle execution: one instruction completes in one clock cycle, although it performs multiple logical activities. For lw: instruction fetch → register read → ALU address calculation → data-memory read → register write-back, all within one cycle.

$$T_{clock} \geq \text{longest required instruction-path delay}$$

In the basic slide implementation, **lw determines the critical path**. If lw needs 700 ps and add needs 500 ps, both still use a 700 ps fixed cycle. Thus, **CPI = 1 does not guarantee high performance**.

TRAP

- PC contains an instruction **address**, not the instruction.
- PC + 4 follows from **4-byte instructions and byte addressing**.
- Single-cycle does not mean that all operations happen simultaneously.
- The clock must allow the result to become ready before it is captured.
- Register reading and register writing are different actions.

L3 · Register Connections and Basic Controls

MIPS has 32 registers, so register numbers require **5 bits**. Register values are **32 bits**.

Instruction	rs	rt	Destination
add rd, rs, rt	First operand	Second operand	rd
lw rt, offset(rs)	Base address	Destination	rt
sw rt, offset(rs)	Base address	Data to store	None

Standard field positions: rs = Instruction[25:21]; rt = Instruction[20:16]; rd = Instruction[15:11]; immediate = Instruction[15:0].

Control	0	1
RegDst	Destination = rt	Destination = rd
ALUSrc	Second ALU input = register value	Second ALU input = extended immediate
MemtoReg	Write-back = ALU result	Write-back = memory output
RegWrite	No register write	Enable register write
MemRead	No data-memory read	Enable data-memory read
MemWrite	No data-memory write	Enable data-memory write

MEMORIZE **RegDst = where to write; MemtoReg = what to write; RegWrite = whether to write.**

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite
add	1	0	0	1	0	0
lw	0	1	1	1	1	0
sw	X	1	X	0	0	1

X means either value is acceptable because the selection is unused. For sw, RegWrite = 0 makes RegDst and MemtoReg irrelevant.

Example: lw $t0, 8($s0) selects rt as the destination, the offset as the second ALU input, and memory data for write-back. **Diagrams/reference:** slides 21, 25 and 27 (Figs. 3.3, 3.2, 3.4).

L4 · ALUOp and ALU Control

```
Opcode → Main control → ALUOp
ALUOp + funct → ALU control unit → 4-bit ALU control → ALU
```

- **ALUOp:** two-bit instruction-category signal.
- **ALU control:** four-bit signal selecting the actual ALU operation.
- For R-type instructions, the funct field selects the specific operation.

Instruction class	ALUOp	Operation
lw, sw	00	ADD
beq	01	SUBTRACT
R-type	10	Decode funct

R-type instruction	funct	ALU control
add	100000	0010
sub	100010	0110
and	100100	0000
or	100101	0001
slt	101010	0111

NOR's ALU control code is **1100**.

Basic main-control table

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type	1	0	0	1	0	0	0	10
lw	0	1	1	1	1	0	0	00
sw	X	1	X	0	0	1	0	00

Supported R-type instructions share these main-control values; funct determines the particular ALU operation.

Trace method: fields → register reads → ALU inputs/operation → memory → write-back → next PC.

Example — sub $t2, $t0, $t1, where $t0 = 50, $t1 = 12: ALUOp = 10; funct = 100010 → ALU control = 0110. ALU computes 50 − 12 = 38. RegDst = 1 selects rd = $t2; MemtoReg = 0 selects the ALU result; RegWrite = 1 writes $t2 = 38.

TRAP

- ALUOp = 10 means **decode funct**, not a particular arithmetic operation.
- add and lw both use ALU control **0010**, but have different ALUOp values.
- MemtoReg selects a value; RegWrite enables the write.
- Use the ALU codes specified by the question or given datapath.

Slide references: 23–24 for control codes; 28–29 for instruction traces (Figs. 4.1–4.2).

L5 · beq Datapath and Control

beq rs, rt, Target — branch when R[rs] = R[rt]. Both rs and rt are source registers; no register is written.

Target = PC + 4 + 4 × signed immediate
PCSrc = Branch AND Zero

- Equal registers → subtraction result 0 → Zero = 1 → select target.
- Unequal registers → Zero = 0 → select PC + 4.

Hardware

1. Main ALU subtracts the two register values.
2. Sign extender and shift-left-2 convert the immediate to a byte displacement.
3. A separate adder adds the displacement to PC + 4.
4. Branch AND Zero controls the next-PC mux.

RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp	ALU control
X	0	X	0	0	0	1	01	0110

The control row is the same whether the branch is taken or not. **Zero and PCSrc depend on the compared values.**

Example — PC = 400, immediate = 3: PC + 4 = 404; displacement = 3 × 4 = 12; target = 416. Equal registers → next PC = 416. Unequal registers → next PC = 404.

TRAP

- Use **PC + 4**, not PC, as the base.
- Multiply the branch immediate by 4; do not do this for lw/sw.
- ALUSrc = 0 because the main ALU compares two registers.
- Branch = 1 even for an untaken beq.
- ALU result, branch target and actual next PC are different quantities.

Diagrams: slide 19 for branch components (Fig. 5.1); slide 22 for their integration (Fig. 5.2); slide 30 for the branch execution trace (Fig. 5.3).

L6 · Jump Instruction

j Target — unconditionally changes PC to the jump target. J-format: **6-bit opcode + 26-bit target field**; j opcode: **000010**.

Jump address = { (PC + 4)[31:28], Instruction[25:0], 00 }

- Shift the 26-bit field left by 2 → 28 bits.
- Prepend the upper four bits of PC + 4 → 32 bits.
- This is concatenation, not PC-relative addition.

```
PC     = 0x20000020          Field     = 0x000000C
PC + 4 = 0x20000024  → upper hex digit = 2
Field × 4 = 0x0000030  →  Next PC = 0x20000030
```

Hardware additions: shift-left-2 wiring for Instruction[25:0]; concatenation with (PC + 4)[31:28]; new mux before PC; Jump control decoded from opcode. **Jump = 0:** existing sequential/branch path. **Jump = 1:** jump target. No additional target-calculation adder is required.

Control: for j — Jump = 1; RegWrite = 0; MemWrite = 0; MemRead = 0; Branch = 0. RegDst, ALUSrc, MemtoReg and ALUOp are don't-cares because their results are unused.

Reachability — a direct j requires a word-aligned target and target upper four bits equal to those of **PC + 4**. For a reachable target:

$$\text{Encoded field} = (\text{Target address AND } 0x0FFFFFFF) / 4$$

TRAP

- Using PC instead of PC + 4 at a region boundary.
- Adding the shifted target to PC + 4.
- Appending two hexadecimal zeros instead of two binary zeros.
- Assuming j writes a register — it does not.

Memorize: formula on slide 31, corrected to PC + 4; added datapath on **slide 32** (Fig. 6.2).

L7 · Full Datapath and Single-Cycle Performance

Diagram construction order

1. PC, instruction memory, register file, ALU and data memory.
2. RegDst, ALUSrc and MemtoReg muxes.
3. Direct Read data 2 → memory Write data connection.
4. PC + 4, branch target, branch-selection mux and jump-selection mux.
5. Main control, ALU control and enable signals.

Practise: slide 26 without jump (Fig. 7.1); slide 32 with jump (Fig. 6.2).

Control	Input 0	Input 1
RegDst	rt	rd
ALUSrc	Register Read data 2	Sign-extended immediate
MemtoReg	ALU result	Memory output

For restricted instruction sets, remove unused paths. A mux can be removed when only one of its choices is needed.

Timing formulas (single-cycle, CPI = 1):

$$T_{clock} \geq \max(\text{required instruction-path delays})$$

$$T_{execution} = N \times T_{clock}$$

$$f_{max} = 1 / T_{clock} \qquad f\ (\text{GHz}) = 1000 / T\ (\text{ps})$$

Critical path — lw is the critical instruction in the basic slide design: instruction memory → register read → ALU → data-memory read → register write-back. Example: if the longest path is 800 ps, every instruction takes 800 ps; a sequence of 100 instructions takes 80 ns.

Short theory answers

- **Single-cycle inefficiency:** every instruction uses the long cycle required by the slowest instruction.
- **Separate memories:** instruction fetch and load/store data access must both be supported in one cycle; separate memories avoid a single-port resource conflict.
- **Separate PC + 4 adder:** allows PC advancement while the main ALU performs the instruction's operation.
- **Timing calculations:** follow the stated delay model and include only relevant path delays.

L8A · Datapath Modifications: bne, jr, jal

Method: write the required operation → find the missing connection → add mux/control → specify control values. Use slide 32 as the base datapath.

bne — same subtraction and target calculation as beq, but branch when Zero = 0. Define BNE = 0 for beq, BNE = 1 for bne, and Branch = 1 for both:

$$\textbf{PCSrc = Branch AND (Zero XOR BNE)}$$

Add an XOR gate and BNE control. Other controls match beq.

jr — PC ← R[rs]. Add a PC-input mux selecting **0:** existing next-PC output, **1:** register-file Read data 1. Assert JR = 1; set RegWrite = 0 and MemWrite = 0. Decode opcode 000000 and funct 001000. No shift or concatenation is required.

jal — in the simplified no-delay-slot model: **R[31] ← PC + 4, PC ← jump target**. Reuse the j path. Add:

1. Destination-register mux selecting constant 31.
2. Write-back mux selecting PC + 4.

Assert Link = 1, Jump = 1, RegWrite = 1. No new adder is needed. Example: jal at address 400 targeting 800 writes $ra = 404 and sets PC = 800.

MEMORIZE

- **bne:** Zero XOR BNE, then AND with Branch.
- **jr:** Read data 1 → added PC mux.
- **jal:** constant 31 → destination mux; PC + 4 → write-data mux.

L8B · slt, lui and Changed Formats

slt rd, rs, rt — writes 1 if signed R[rs] < R[rt]; otherwise writes 0.

- Normal R-type datapath.
- RegDst = 1, ALUSrc = 0, MemtoReg = 0, RegWrite = 1.
- ALUOp = 10; funct = **101010**; ALU control = **0111**.
- No branch or data-memory access.
- If ALU already supports SLT, no new datapath wiring is needed.

lui rt, imm16 —

$$\text{R[rt]} \leftarrow \{\ \text{imm16, 16'b0}\ \}$$

Example: lui $t0, 0x1234 → $t0 = 0x12340000. Add wiring that appends 16 zeros and a mux selecting this value for register write-back.

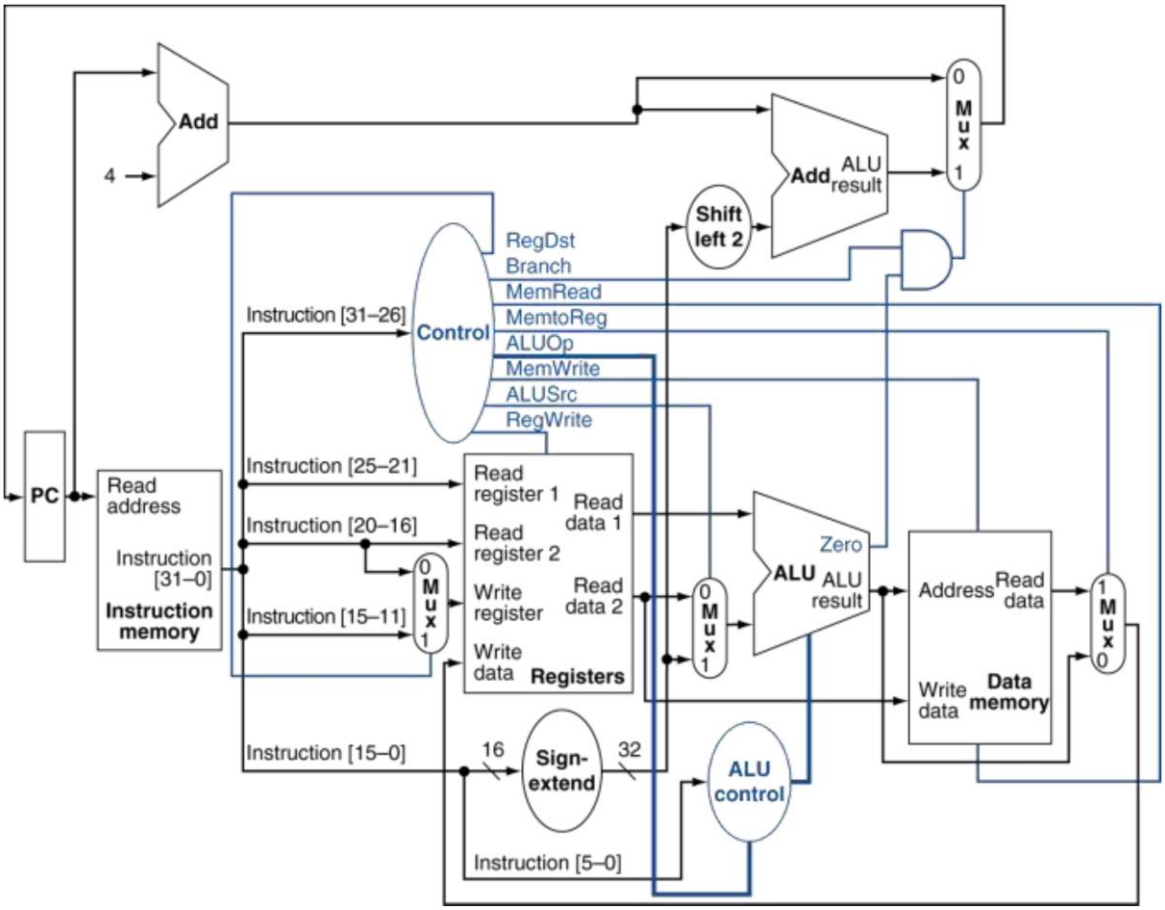
- Destination = rt; RegDst = 0.
- RegWrite = 1; LUI = 1.
- No data-memory access.
- Opcode = **001111**.
- In this implementation, ALU output and the old MemtoReg selection are unused.
- The old lower 16 register bits are replaced with zeros.

Modified-ISA questions

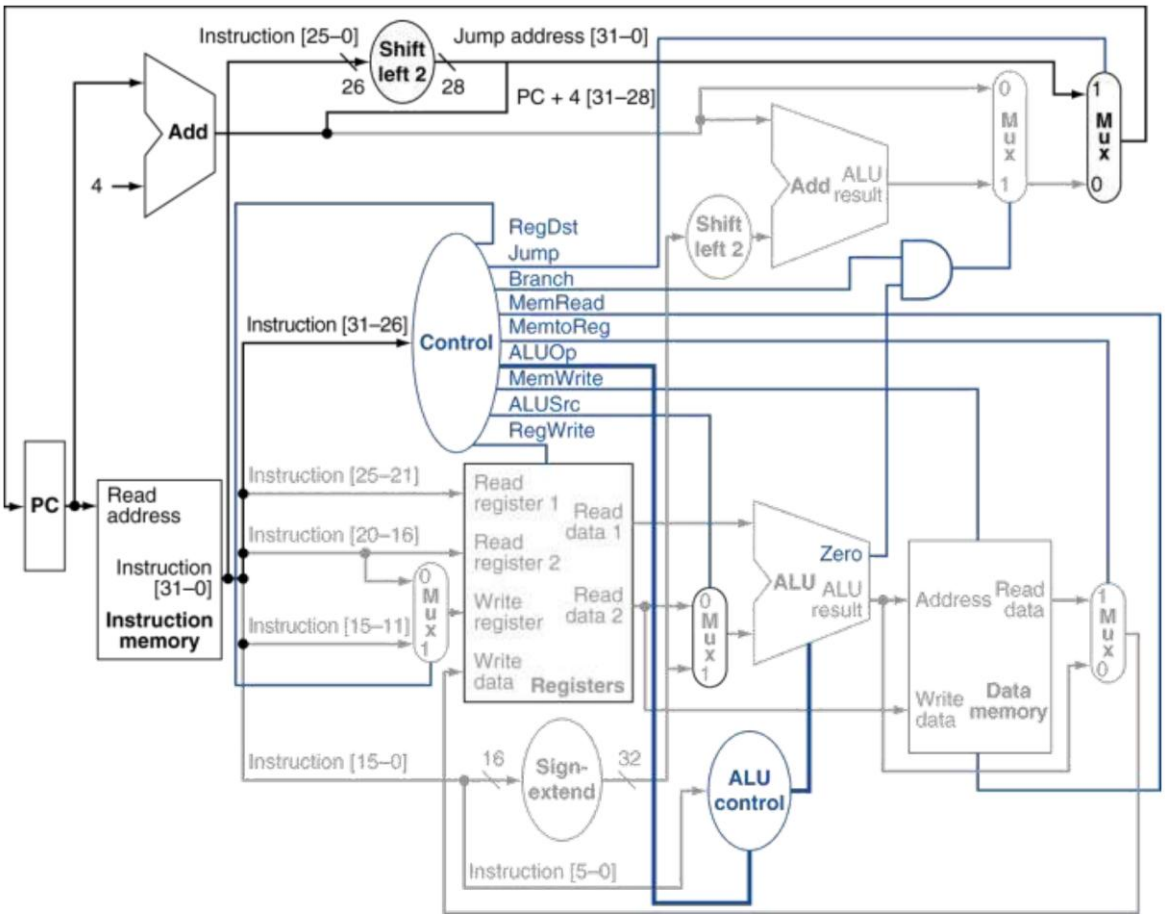
1. Identify each field's meaning and position.
2. Map first source/base → Read register 1.
3. Map second source/store data → Read register 2.
4. Map destination → Write register.
5. Compare formats: same field position → wire; different positions → mux.
6. Label register-number connections as **5 bits**.

TRAP Do not assume standard MIPS field positions when the question supplies different ones.

Key Diagrams to Reproduce



Datapath with control, without jump (slide 26) — same as Fig. 7.1



Datapath with jumps added (slide 32) — same as Fig. 6.2; base for all modification questions